

A Brain-Friendly Guide to OOA&D

Head First Object-Oriented Analysis & Design



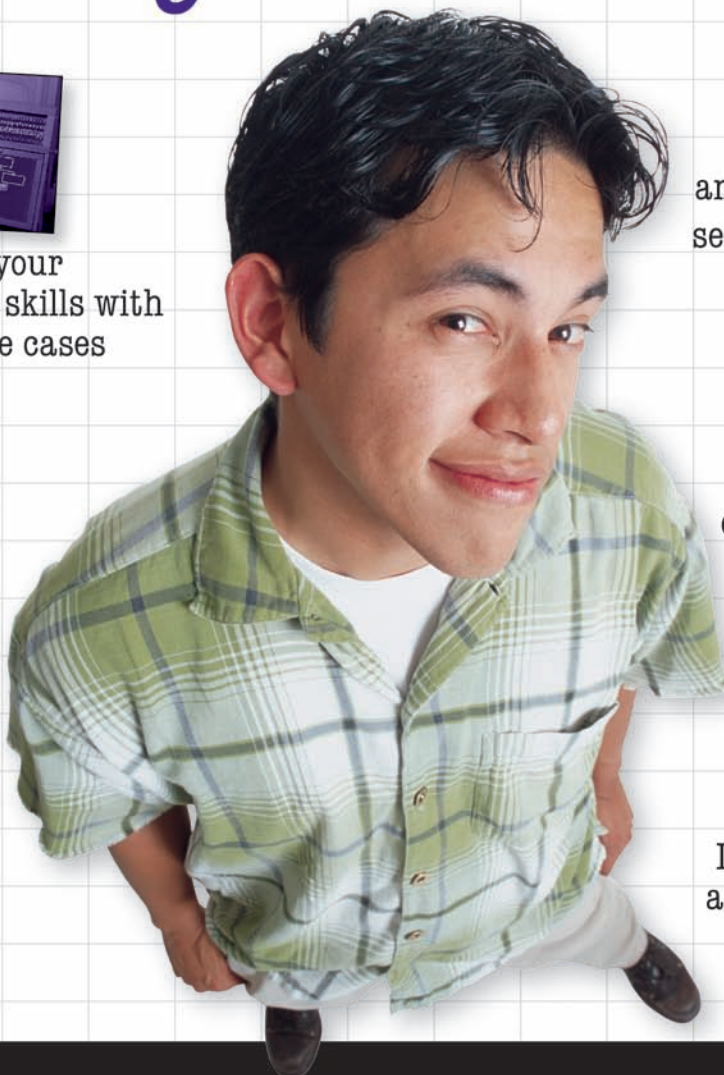
Improve your
communication skills with
UML and use cases



Bend your mind
around dozens
of OO exercises



Avoid leaving
your customers
unsatisfied



Turn your
requirements
and designs into
serious software



Load important OO
design principles straight
into your brain



Discover how abstraction,
aggregation, and delegation
helped Mary get around
Objectville

O'REILLY®

Brett D. McLaughlin, Gary Pollice & David West

Head First Object-Oriented Analysis & Design

Software Development/Software Engineering

"Head First Object-Oriented Analysis and Design is a refreshing look at subject of OOA&D. What sets this book apart is its focus on learning. The authors have made the content of OOA&D accessible, usable for the practitioner."

—Ivar Jacobson
Ivar Jacobson Consulting

"Hidden behind the funny pictures and crazy fonts is a serious, intelligent, extremely well-crafted presentation of OO Analysis and Design. As I read the book, I felt like I was looking over the shoulder of an expert designer who was explaining to me what issues were important at each step, and why."

—Edward Sciore
Associate Professor
Computer Science Department, Boston College

"I just finished reading HF OOA&D and I loved it! The thing I liked most about this book was its focus on why we do OOA&D—to write great software!"

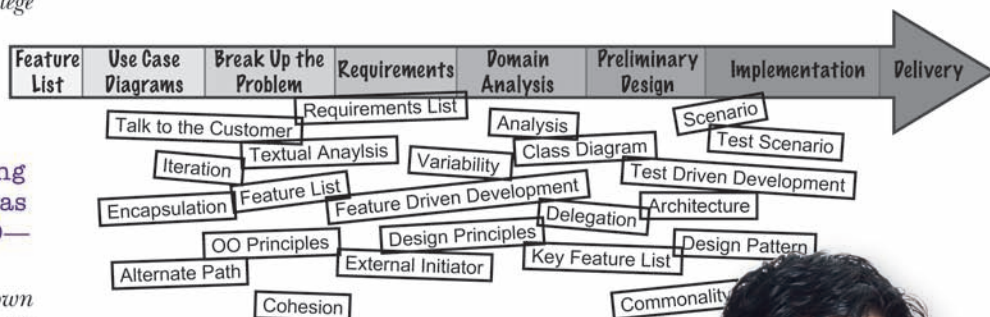
—Kyle Brown
Distinguished Engineer, IBM

Tired of reading Object-Oriented Analysis and Design books that only make sense after you're an expert? You've probably heard that OOA&D can help you write great software every time—software that makes your boss happy, and your customers satisfied.

But how?

Head First Object-Oriented Analysis & Design shows you how to analyze, design, and write serious object-oriented software: software that's easy to reuse, maintain, and extend; software that doesn't hurt your head; software that lets you add new features without breaking the old ones. Inside you will learn how to:

- Use OO principles like encapsulation and delegation to build applications that are flexible.
- Apply the Open-Closed Principle (OCP) and the Single Responsibility Principle (SRP) to promote reuse of your code.
- Learn how OO principles, design patterns, and different development approaches all fit into the OOA&D project lifecycle.
- Use UML, use cases, and diagrams to ensure that all stakeholders are communicating clearly to help you deliver the right software that meets everyone's needs.



By exploiting how your brain works, *Head First OOA&D* compresses the time it takes to learn and retain complex information. Expect to have fun, expect to learn, expect to be writing great software consistently by the time you're finished reading this!

www.oreilly.com

US \$49.99

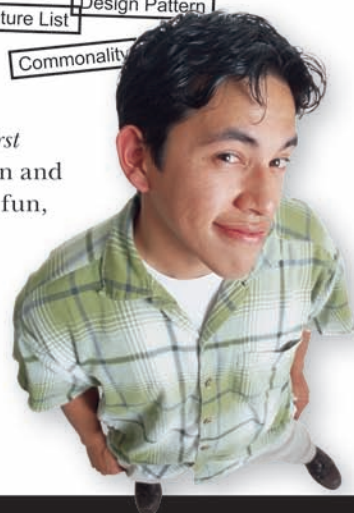
CAN \$64.99

ISBN-10: 0-596-00867-8

ISBN-13: 978-0-596-00867-3



O'REILLY®



Praise for *Head First OOA&D*

“*Head First Object-Oriented Analysis and Design* is a refreshing look at the subject of OOA&D. What sets this book apart is its focus on learning. There are too many books on the market that spend a lot of time telling you why, but do not actually enable the practitioner to start work on a project. Those books are very interesting, but not very practical. I strongly believe that the future of software development practice will focus on the practitioner. The authors have made the content of OOA&D accessible and usable for the practitioner.”

— **Ivar Jacobson, Ivar Jacobson Consulting**

“I just finished reading *HF OOA&D*, and I loved it! The book manages to get across the essentials of object-oriented analysis and design with UML and use cases, and even several lectures on good software design, all in a fast-paced, easy to understand way. The thing I liked most about this book was its focus on why we do OOA&D—to write great software! By defining what great software is and showing how each step in the OOA&D process leads you towards that goal, it can teach even the most jaded Java programmer why OOA&D matters. This is a great ‘first book’ on design for anyone who is new to Java, or even for those who have been Java programmers for a while but have been scared off by the massive tomes on OO Analysis and Design.”

— **Kyle Brown, Distinguished Engineer, IBM**

“Finally a book on OOA&D that recognizes that the UML is just a notation and that what matters when developing software is taking the time to think the issues through.”

— **Pete McBreen, Author, *Software Craftsmanship***

“The book does a good job of capturing that entertaining, visually oriented, ‘Head First’ writing style. But hidden behind the funny pictures and crazy fonts is a serious, intelligent, extremely well-crafted presentation of OO Analysis and Design. This book has a strong opinion of how to design programs, and communicates it effectively. I love the way it uses running examples to lead the reader through the various stages of the design process. As I read the book, I felt like I was looking over the shoulder of an expert designer who was explaining to me what issues were important at each step, and why.”

— **Edward Sciore, Associate Professor, Computer Science Department
Boston College**

“This is a well-designed book that delivers what it promises to its readers: how to analyze, design, and write serious object-oriented software. Its contents flow effortlessly from using use cases for capturing requirements to analysis, design, implementation, testing, and iteration. Every step in the development of object-oriented software is presented in light of sound software engineering principles. The examples are clear and illustrative. This is a solid and refreshing book on object-oriented software development.”

— **Dung Zung Nguyen, Lecturer
Rice University**

Praise for other *Head First* books by the authors

“When arriving home after a 10-hour day at the office programming, who has the energy to plow through yet another new facet of emerging technology? If a developer is going to invest free time in self-driven career development, should it not be at least remotely enjoyable? Judging from the content of O’Reilly’s new release *Head Rush Ajax*, the answer is yes...*Head Rush Ajax* is a most enjoyable launchpad into the world of Ajax web applications, well worth the investment in time and money.”

— **Barry Hawkins, Slashdot.org**

“By starting with simple concepts and examples, the book gently takes the reader from humble beginnings to (by the end of the book) where the reader should be comfortable creating Ajax-based websites... Probably the best web designer centric book on Ajax.”

— **Stefan Mischook, Killersites.com**

“Using the irreverent style common of the *Head First/Head Rush* series of books, this book starts at the beginning and introduces you to all you need to know to be able to write the JavaScript that will both send requests to the server and update the page with the results when they are returned...One of the best things about this book (apart from the excellent explanations of how the code works) is that it also looks at security issues...If you learn Ajax from this book you are unlikely to forget much of what you learn.”

— **Stephen Chapman, JavaScript.About.com**

“*Head Rush Ajax* is the book if you want to cut through all the hype and learn how to make your web apps sparkled...your users will love you for it!”

— **Kristin Stromberg, Aguirre International**

“If you know some HTML, a dollop of CSS, a little JavaScript, and a bit of PHP, but you’re mystified about what all the Ajax hype is about, this book is for you...You’ll have a blast learning Ajax with *Head Rush Ajax*. By the time you’ve reached the end of the book, all those web technologies that didn’t quite fit together in your head will all snap into place and you’ll have The Ajax Power! You’ll know the secrets behind some of the most popular web applications on the Internet. You’ll impress your friends and co-workers with you knowledge of how those interactive maps and web forms really work.”

— **Elisabeth Freeman, The Walt Disney Internet Group**
Co-Author, *Head First Design Patterns* and *Head First HTML with CSS & XHTML*

“If you thought Ajax was rocket science, this book is for you. *Head Rush Ajax* puts dynamic, compelling experiences within reach for every web developer.”

— **Jesse James Garrett, Adaptive Path**

“This stuff is brain candy; I can’t get enough of it.”

— **Pauline McNamara, Center for New Technologies and Education**
Fribourg University, Switzerland

Praise for other *Head First* Books

“I *heart* *Head First HTML with CSS & XHTML* – it teaches you everything you need to learn in a ‘fun coated’ format!”

— **Sally Applin, UI Designer and Fine Artist, <http://sally.com>.**

“My wife stole the book. She’s never done any web design, so she needed a book like *Head First HTML with CSS & XHTML* to take her from beginning to end. She now has a list of web sites she wants to build – for our son’s class, our family, ... If I’m lucky, I’ll get the book back when she’s done.”

— **David Kaminsky, Master Inventor, IBM**

“Freeman’s *Head First HTML with CSS & XHTML* is a most entertaining book for learning how to build a great web page. It not only covers everything you need to know about HTML, CSS, and XHTML, it also excels in explaining everything in layman’s terms with a lot of great examples. I found the book truly enjoyable to read, and I learned something new!”

— **Newton Lee, Editor-in-Chief, ACM Computers in Entertainment
<http://www.acmcie.org>**

From the awesome *Head First Java* folks, this book uses every conceivable trick to help you understand and remember. Not just loads of pictures: pictures of humans, which tend to interest other humans. Surprises everywhere. Stories, because humans love narrative. (Stories about things like pizza and chocolate. Need we say more?) Plus, it’s darned funny.

— **Bill Camarda, READ ONLY**

“This book’s admirable clarity, humor and substantial doses of clever make it the sort of book that helps even non-programmers think well about problem-solving.”

— **Cory Doctorow, co-editor of Boing Boing
Author, “Down and Out in the Magic Kingdom”
and “Someone Comes to Town, Someone Leaves Town”**

“I feel like a thousand pounds of books have just been lifted off of my head.”

— **Ward Cunningham, inventor of the Wiki
and founder of the Hillside Group**

“I literally love this book. In fact, I kissed this book in front of my wife.”

— **Satish Kumar**

Other related books from O'Reilly

Practical Development Environments
Process Improvement Essentials
Prefactoring
Ajax Design Patterns
Learning UML
Applied Software Project Management
The Art of Project Management
UML 2.0 in a Nutshell
Unit Test Frameworks

Other books in O'Reilly's *Head First* Series

Head First Design Patterns
Head First Java
Head First Servlets and JSP
Head First EJB
Head First HTML with CSS & XHTML
Head Rush Ajax
Head First OOA&D
Head First PMP (2007)
Head First Algebra (2007)
Head First Software Development (2007)

Head First Object-Oriented Analysis and Design

Wouldn't it be dreamy
if there was an analysis and
design book that was more fun
than going to an HR benefits
meeting? It's probably nothing
but a fantasy...



Brett D. McLaughlin
Gary Pollice
David West

O'REILLY®

Beijing • Cambridge • Köln • Sebastopol • Tokyo

Head First Object-Oriented Analysis and Design

by Brett D. McLaughlin, Gary Pollice, and David West

Copyright © 2007 O'Reilly Media, Inc. All rights reserved.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly Media books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (*safari.oreilly.com*). For more information, contact our corporate/institutional sales department: (800) 998-9938 or *corporate@oreilly.com*.

Series Creators:	Kathy Sierra, Bert Bates
Series Editor:	Brett D. McLaughlin
Editor:	Mary O'Brien
Cover Designer:	Mike Kohnke, Edie Freedman
OO:	Brett D. McLaughlin
A:	David West
D:	Gary Pollice
Page Viewer:	Dean and Robbie McLaughlin

Printing History:

November 2006: First Edition.



The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. The *Head First* series designations, *Head First OOA&D*, and related trade dress are trademarks of O'Reilly Media, Inc.

Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in this book, and O'Reilly Media, Inc., was aware of a trademark claim, the designations have been printed in caps or initial caps.

While every precaution has been taken in the preparation of this book, the publisher and the authors assume no responsibility for errors or omissions, or for damages resulting from the use of the information contained herein.

In other words, if you use anything in *Head First OOA&D* to, say, write code that controls an American space shuttle, you're on your own.

No dogs, rabbits, or woodchucks were harmed in the making of this book, or Todd and Gina's dog door.



This book uses RepKover™, a durable and flexible lay-flat binding.

ISBN: 978-0-596-00867-3

[M]

[2011-03-04]

To all the brilliant people who came up with various ways to gather requirements, analyze software, and design code...

...thanks for coming up with something good enough to produce great software, but hard enough that we needed this book to explain it all.

Brett McLaughlin is a guitar player who is still struggling with the realization that you can't pay the bills if you're into acoustic fingerstyle blues and jazz. He's just recently discovered, to his delight, that writing books that help people become better programmers does pay the bills. He's very happy about this, as are his wife Leigh, and his kids, Dean and Robbie.

Before Brett wandered into Head First land, he developed enterprise Java applications for Nextel Communications and Allegiance Telecom. When that became fairly mundane, Brett took on application servers, working on the internals of the Lutris Enhydra servlet engine and EJB container. Along the way, Brett got hooked on open source software, and helped found several cool programming tools, like Jakarta Turbine and JDOM. Write to him at brett@oreilly.com.

Brett →



Gary →

Gary Pollice is a self-labeled curmudgeon (that's a crusty, ill-tempered, usually old man) who spent over 35 years in industry trying to figure out what he wanted to be when he grew up. Even though he hasn't grown up yet, he did make the move in 2003 to the hallowed halls of academia where he has been corrupting the minds of the next generation of software developers with radical ideas like, "develop software for your customer, learn how to work as part of a team, design and code quality and elegance and correctness counts, and it's okay to be a nerd as long as you are a great one."

Gary is a Professor of Practice (meaning he had a real job before becoming a professor) at Worcester Polytechnic Institute. He lives in central Massachusetts with his wife, Vikki, and their two dogs, Aloysius and Ignatius. You can visit his WPI home page at <http://web.cs.wpi.edu/~gpollice/>. Feel free to drop him a note and complain or cheer about the book.

Dave West would like to describe himself as sheik geek. Unfortunately no one would describe him in that way. They would say he is a professional Englishman who likes to talk about software development best practices with the passion and energy of an evangelical preacher. Recently Dave has moved to Ivar Jacobson Consulting, where he runs the Americas and can combine his desire to talk about software development and spread the word on rugby and football, and argue that cricket is more exciting than baseball.

Before running the Americas for Ivar Jacobson Consulting, Dave worked for a number of years at Rational Software (now a part of IBM). Dave held many positions at Rational and then IBM, including Product Manager for RUP where he introduced the idea of process plug-ins and agility to RUP. Dave can be contacted at dwest@ivarjacobson.com.

Dave →



Table of Contents (summary)

	Intro	xxiii
1	Great Software Begins Here: <i>well-designed apps rock</i>	1
2	Give Them What They Want: <i>gathering requirements</i>	55
3	I Love You, You're Perfect... Now Change: <i>requirements change</i>	111
4	Taking Your Software Into the Real World: <i>analysis</i>	145
5	Part 1: Nothing Ever Stays the Same: <i>good design</i>	197
	Interlude: OO CATASTROPHE	221
	Part 2: Give Your Software a 30-minute Workout: <i>flexible software</i>	233
6	"My Name is Art Vandelay": <i>solving really big problems</i>	279
7	Bringing Order to Chaos: <i>architecture</i>	323
8	Originality is Overrated: <i>design principles</i>	375
9	The Software is Still for the Customer: <i>iteration and testing</i>	423
10	Putting It All Together: <i>the ooa&d lifecycle</i>	483
	Appendix I: <i>leftovers</i>	557
	Appendix II: <i>welcome to objectville</i>	575

Table of Contents (the real thing)

Intro

Your brain on OOA&D. Here *you* are trying to *learn* something, while here your *brain* is doing you a favor by making sure the learning doesn't *stick*. Your brain's thinking, "Better leave room for more important things, like which wild animals to avoid and whether naked snowboarding is a bad idea." So how *do* you trick your brain into thinking that your life depends on knowing object-oriented analysis and design?

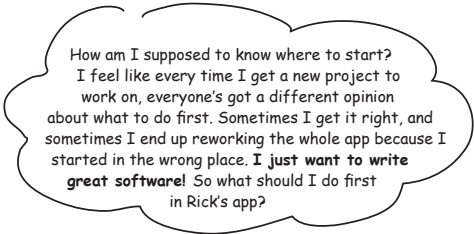
Who is this book for?	xxiv
We know what you're thinking	xxv
Metacognition	xxvii
Bend your brain into submission	xxix
Read Me	xxx
The Technical Team	xxxii
Acknowledgements	xxxiii

well-designed apps rock

1

Great Software Begins Here

So how do you *really* write great software? It's never easy trying to figure out **where to start**. Does the application actually **do what it's supposed to**? And what about things like duplicate code—that can't be good, can it? It's usually pretty hard to know **what you should work on first**, and still make sure you don't screw everything else up in the process. No worries here, though. By the time you're done with this chapter, you'll **know how to write great software**, and be well on your way to improving the way you develop applications forever. Finally, you'll understand why **OOAD** is a four-letter word that your mother actually *wants* you to know about.



Rock and roll is forever!	2
Rick's shiny new application	3
What's the FIRST thing you'd change?	8
Great Software is...	10
Great software in 3 easy steps	13
Focus on functionality first	18
Test drive	23
Looking for problems	25
Analysis	26
Apply basic OO principles	31
Design once, design twice	36
How easy is it to change your applications?	38
Encapsulate what varies	41
Delegation	43
Great software at last (for now)	46
OOA&D is about writing great software	49
Bullet Points	50

gathering requirements

2 Give Them What They Want

Everybody loves a satisfied customer. You already know that the first step in writing great software is making sure it does what the customer wants it to. But how do you figure out **what a customer really wants**? And how do you make sure that the customer even *knows* what they really want? That's where **good requirements** come in, and in this chapter, you're going to learn how to **satisfy your customer** by making sure what you deliver is actually what they asked for. By the time you're done, all of your projects will be "satisfaction guaranteed," and you'll be well on your way to writing great software, every time.

Todd and Gina's Dog Door, version 2.0

Requirements List

Todd and Gina's Dog Door, version 2.0

What the Door Does

1. The tall

2. A b

3. On

4. The dog door opens.

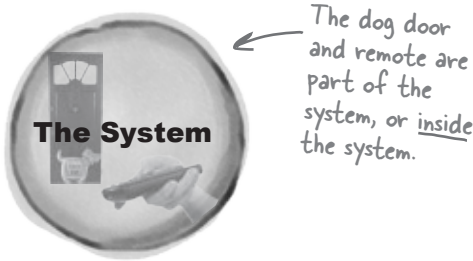
5. Fido goes outside.

6. Fido does his business.

7. Fido goes back inside.

8. The door shuts automatically.

You've got a new programming gig	56
Test drive	59
Incorrect usage (sort of)	61
What is a requirement?	62
Creating a requirements list	64
Plan for things going wrong	68
Alternate paths handle system problems	70
Introducing use cases	72
One use case, three parts	74
Check your requirements against your use cases	78
Your system must work in the real world	85
Getting to know the Happy Path	92
OOA&D Toolbox	106



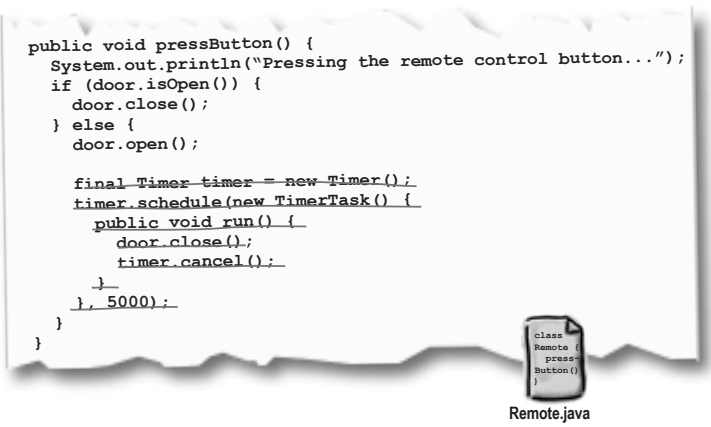
requirements change

3

I Love You, You're Perfect... Now Change

Think you've got just what the customer wanted? Not so fast... So you've talked to your customer, gathered requirements, written out your use cases, and delivered a killer application. It's time for a nice relaxing cocktail, right? Right... until your customer decides that they really wanted something different than what they told you. They love what you've done, really, but it's not quite good enough anymore. In the real world, requirements are always changing, and it's up to you to roll with these changes and keep your customer satisfied.

You're a hero!	112
You're a goat!	113
The one constant in software analysis & design	115
Original path? Alternate path? Who can tell?	120
Use cases have to make sense to you	122
Start to finish: a single scenario	124
Confessions of an Alternate Path	126
Finishing up the requirements list	130
Duplicate code is a bad idea	138
Final test drive	140
Write your own design principle	141
OOA&D Toolbox	142



analysis

4

Taking Your Software into the Real World**It's time to graduate to real-world applications.**

Your application has to do more than work on your own personal development machine, finely tuned and perfectly setup; your apps have to work when **real people use them**.

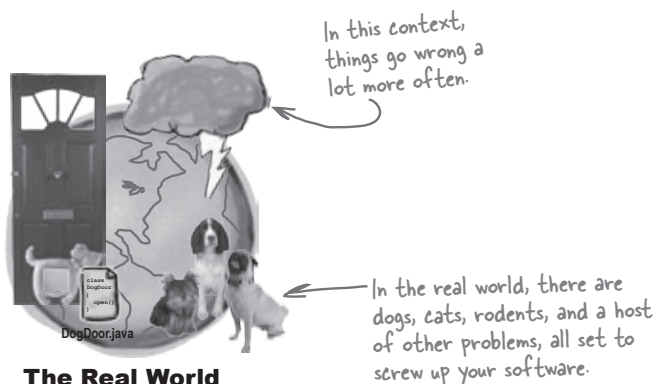
This chapter is all about making sure that your software works in a **real-world context**.

You'll learn how **textual analysis** can take that use case you've been working on and turn it into classes and methods that you know are what your customers want. And when you're done, you too can say: "I did it! My software is **ready for the real world!**"

Once I knew the classes and operations that I needed, I went back and updated my class diagram.



One dog, two dog, three dog, four...	146
Your software has a context	147
Identify the problem	148
Plan a solution	149
A tale of two coders	156
Delegation Detour	160
The power of loosely coupled applications	162
Pay attention to the nouns in your use case	167
From good analysis to good classes...	180
Class diagrams dissected	182
Class diagrams aren't everything	187
Bullet Points	191



5 (part 1)

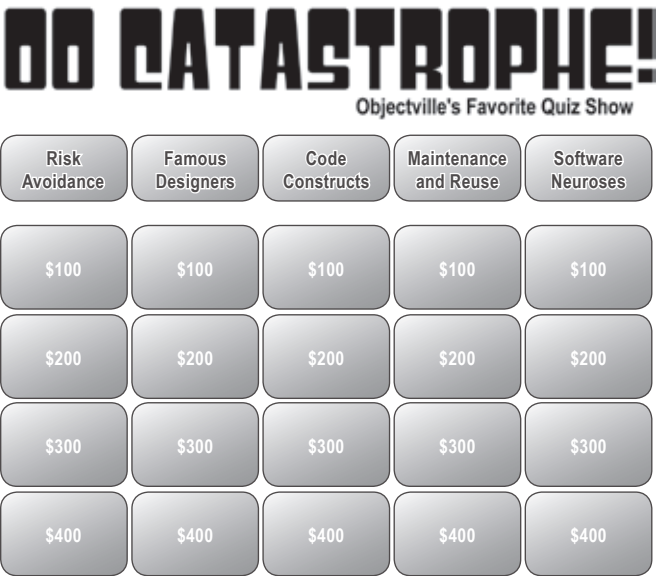
good design = flexible software

Nothing Ever Stays the Same

Change is inevitable. No matter how much you like your software right now, it's probably going to **change** tomorrow. And the harder you make it for your software to change, the more difficult it's going to be to respond to your **customer's changing needs**. In this chapter, we're going to revisit an old friend, try and improve an existing software project, and see how **small changes can turn into big problems**. In fact, we're going to uncover a problem so big that it will take a TWO-PART chapter to solve it!

Rick's Guitars is expanding	198
Abstract classes	201
Class diagrams dissected (again)	206
UML Cheat Sheet	207
Design problem tipoffs	213
3 steps to great software (revisited)	215

5 (interlude)



5

(part 2)

good design = flexible software

Give Your Software a 30-minute Workout

Ever wished you were just a bit more flexible?

When you run into problems making changes to your application, it probably means that your software needs to be **more flexible and resilient**. To help stretch your application out, you're going to do some analysis, a whole lot of design, and learn how OO principles can really **loosen up your application**. And for the grand finale, you'll see how **higher cohesion can really help your coupling**. Sound interesting? Turn the page, and let's get back to fixing that inflexible application.

Back to Rick's search tool	234
A closer look at the search() method	237
The benefits of analysis	238
Classes are about behavior	241
Death of a design (decision)	246
Turn bad design decisions into good ones	247
"Double encapsulation" in Rick's software	249
Never be afraid to make mistakes	255
Rick's flexible application	258
Test driving well-designed software	261
How easy is it to change Rick's software?	265
The Great Ease-of-Change Challenge	266
A cohesive class does one thing really well	269
The design/cohesion lifecycle	272
Great software is "good enough"	274
OOA&D Toolbox	276

solving really big problems

6

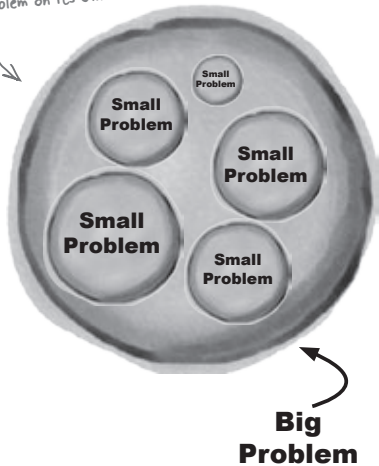
“My Name is Art Vandelay... I am an Architect”

It’s time to build something REALLY BIG. Are you ready?

You’ve got a ton of tools in your OOA&D toolbox, but how do you use those tools when you have to build something **really big**? Well, you may not realize it, but **you’ve got everything you need** to handle big problems. We’ll learn about some new tools, like **domain analysis** and **use case diagrams**, but even these new tools are based on things you already know about—like listening to the customer and understanding what you’re going to build before you start writing code. Get ready... it’s time to start playing the architect.

Solving big problems	280
It’s all in how you look at the big problem	281
Requirements and use cases are a good place to start...	286
Commonality and variability	287
Figure out the features	290
The difference between features and requirements	292
Use cases don’t always help you see the big picture	294
Use case diagrams	296
The Little Actor	301
Actors are people, too (well, not always)	302
Let’s do a little domain analysis	307
Divide and conquer	309
Don’t forget who the customer really is	313
What’s a design pattern?	315
The power of OOA&D (and a little common sense)	318
OOA&D Toolbox	320

This **BIG PROBLEM** is really just a collection of functionalities, where each piece of functionality is really a smaller problem on its own.



7

Bringing Order to Chaos

You have to start somewhere, but you better pick the *right* somewhere! You know how to break your application up into lots of small problems, but all that means is that you have **LOTS** of small problems. In this chapter, we're going to help you figure out **where to start**, and make sure that you don't waste any time working on the wrong things. It's time to take all those **little pieces** laying around your workspace, and figure out how to turn them into a **well-ordered, well-designed application**. Along the way, you'll learn about the all-important **3 Qs of architecture**, and how **Risk** is a lot more than just a cool war game from the '80s.

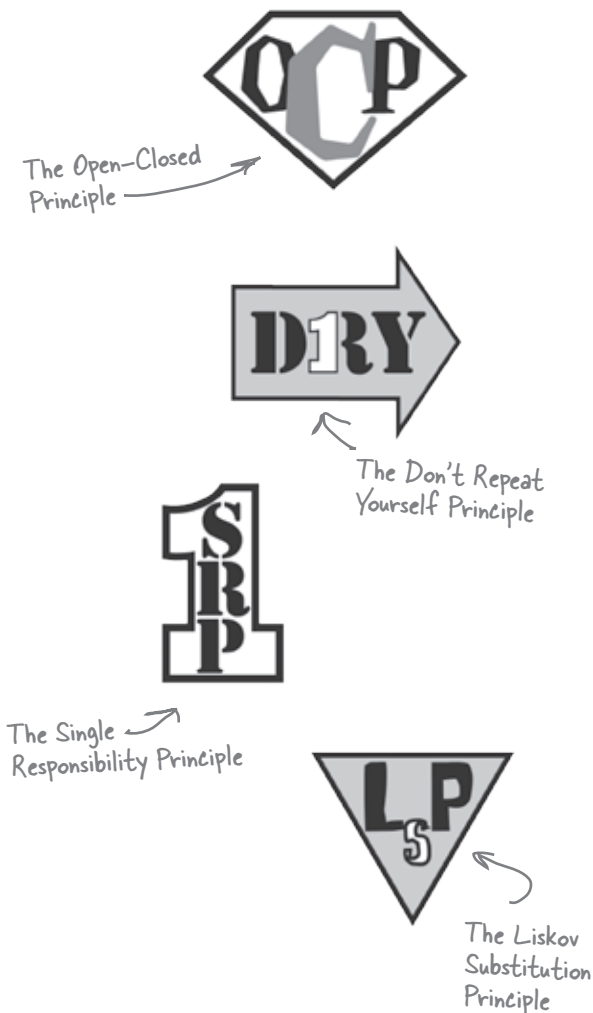


Feeling a little overwhelmed?	324
We need an architecture	326
Start with functionality	329
What's architecturally significant?	331
The three Qs of architecture	332
Reducing risk	338
Scenarios help reduce risk	341
Focus on one feature at a time	349
Architecture is your design structure	351
Commonality revisited	355
Commonality Analysis: the path to flexible software	361
What does it mean? Ask the customer	366
Reducing risk helps you write great software	371
Bullet Points	372

design principles

8 Originality is Overrated

Imitation is the sincerest form of not being stupid. There's nothing as satisfying as coming up with a completely new and original solution to a problem that's been troubling you for days—until you find out someone else **solved the same problem**, long before you did, and did an even better job than you did! In this chapter, we're going to look at some **design principles** that people have come up with over the years, and how they can make you a better programmer. Lay aside your thoughts of “doing it your way”; this chapter is about **doing it the smarter, faster way**.



Design principle roundup	376
The Open-Closed Principle (OCP)	377
The OCP, step-by-step	379
The Don't Repeat Yourself Principle (DRY)	382
DRY is about one requirement in one place	384
The Single Responsibility Principle (SRP)	390
Spotting multiple responsibilities	392
Going from multiple responsibilities to a single responsibility	395
The Liskov Substitution Principle (LSP)	400
Misusing subclassing: a case study in misusing inheritance	401
LSP reveals hidden problems with your inheritance structure	402
Subtypes must be substitutable for their base types	403
Violating the LSP makes for confusing code	404
Delegate functionality to another class	406
Use composition to assemble behaviors from other classes	408
Aggregation: composition, without the abrupt ending	412
Aggregation versus composition	413
Inheritance is just one option	414
Bullet Points	417
OOA&D Toolbox	418

iterating and testing

9

The Software is Still for the Customer

It's time to show the customer how much you really care.

Nagging bosses? Worried clients? Stakeholders that keep asking, "Will it be done on time?" No amount of well-designed code will please your customers; you've got to **show them something working**. And now that you've got a solid OO programming toolkit, it's time to learn how you can **prove to the customer** that your software works. In this chapter, we learn about two ways to **dive deeper** into your software's functionality, and give the customer that warm feeling in their chest that makes them say, **Yes, you're definitely the right developer for this job!**

Unit
type: String
properties: Map
id: int
name: String
weapons: Weapon *
setType(String)
getType(): String
setProperty(String, Object)
getProperty(String): Object
getId(): int
setName(String)
getName(): String
addWeapon(Weapon)
getWeapons(): Weapon *

All the properties that were common across units are represented as variables outside of the properties Map.

Sam figured that id would get set in the Unit constructor, so no need for a setId() method.

Each of the new properties gets its own set of methods.

Your toolbox is filling up	424
You write great software iteratively	426
Iterating deeper: two basic choices	427
Feature driven development	428
Use case driven development	429
Two approaches to development	430
Analysis of a feature	434
Writing test scenarios	437
Test driven development	440
Commonality Analysis (redux)	442
Emphasizing commonality	446
Emphasizing encapsulation	448
Match your tests to your design	452
Test cases dissected...	454
Prove yourself to the customer	460
We've been programming by contract	462
Programming by contract is about trust	463
Defensive programming	464
Break your apps into smaller chunks of functionality	473
Bullet Points	475
OOA&D Toolbox	478

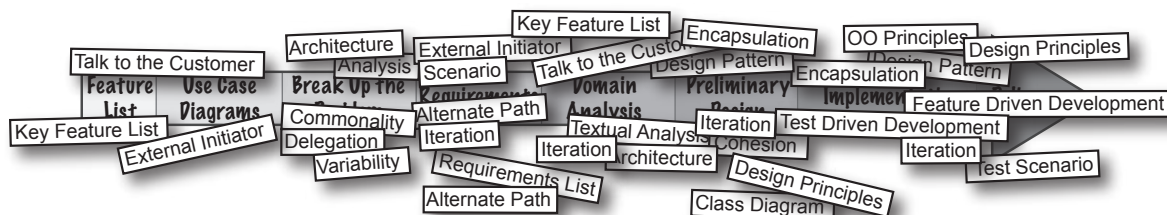
the ooa&d lifecycle

10

Putting It All Together

Are we there yet? We've been working on lots of individual ways to improve your software, but now it's time to **put it all together**. This is it, what you've been waiting for: we're going to take **everything** you've been learning, and show you how it's all really part of **a single process** that you can use over and over again to **write great software**.

Developing software, OOA&D style	484
The Objectville Subway problem	488
Objectville Subway Map	490
Feature lists	493
Use cases reflect usage, features reflect functionality	499
Now start to iterate	503
A closer look at representing a subway	505
To use a Line, or not to use a Line	514
Points of interest on the Objectville Subway (class)	520
Protecting your classes	523
Break time	531
Back to the requirements phase	533
Focus on code, then focus on customers	535
Iteration makes problems easier	539
What does a route look like?	544
Check out Objectville for yourself!	548
Iteration #3, anyone?	551
The journey's not over...	555



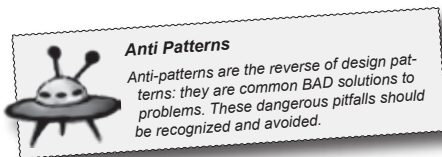
appendix i: leftovers



The Top Ten Topics (we didn't cover)

Believe it or not, there's still more. Yes, with over 550 pages under your belt, there are still things we couldn't cram in. Even though these last ten topics don't deserve more than a mention, we didn't want to let you out of Objectville without a little more information on each one of them. But hey, now you've got just a little bit more to talk about during commercials of CATASTROPHE... and who doesn't love some stimulating OOA&D talk every now and then?

#1. IS-A and HAS-A	558
#2. Use case formats	560
#3. Anti-patterns	563
#4. CRC cards	564
#5. Metrics	566
#6. Sequence diagrams	567
#7. State diagrams	568
#8. Unit testing	570
#9. Coding standards and readable code	572
#10. Refactoring	574



Be sure you write down things that this class does on its own, as well as things it collaborates with other classes on.

Class: DogDoor	
Description: Represents the physical dog door. This provides an interface to the hardware that actually controls the door.	
Responsibilities:	
Name	Collaborator
Open the door	
Close the door	

There's no collaborator class for these.

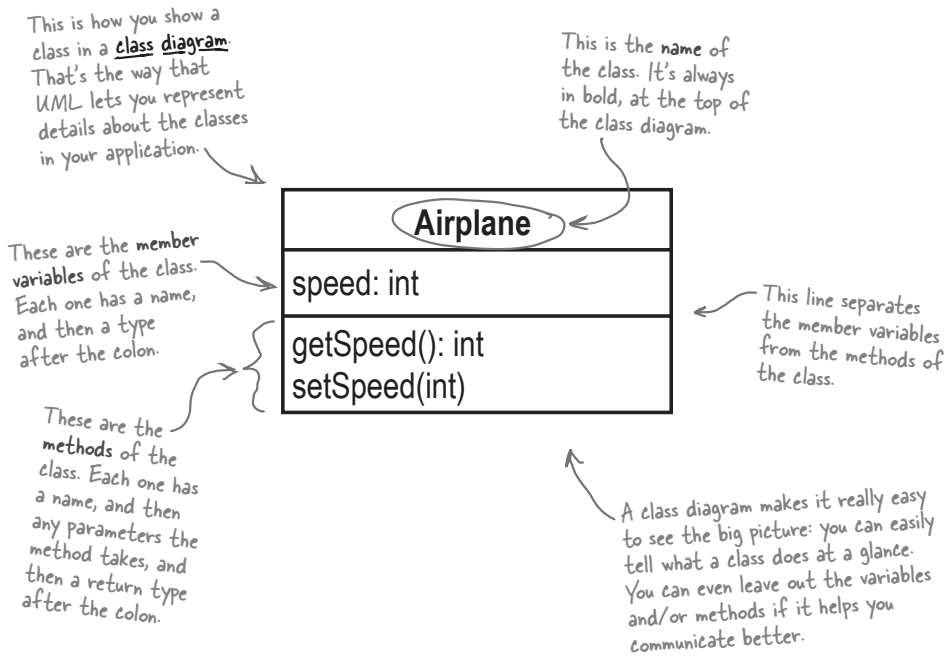
appendix ii: welcome to objectville



Speaking the Language of OO

Get ready to take a trip to a foreign country. It's time to visit Objectville, a land where **objects do just what they're supposed to**, applications are all **well-encapsulated** (you'll find out exactly what that means shortly), and designs are easy to **reuse and extend**. But before we can get going, there are a few things you need to know first, and a little bit of **language skills** you're going to have to learn. Don't worry, though, it won't take long, and before you know it, you'll be speaking the language of OO like you've been living in the well-designed areas of Objectville for years.

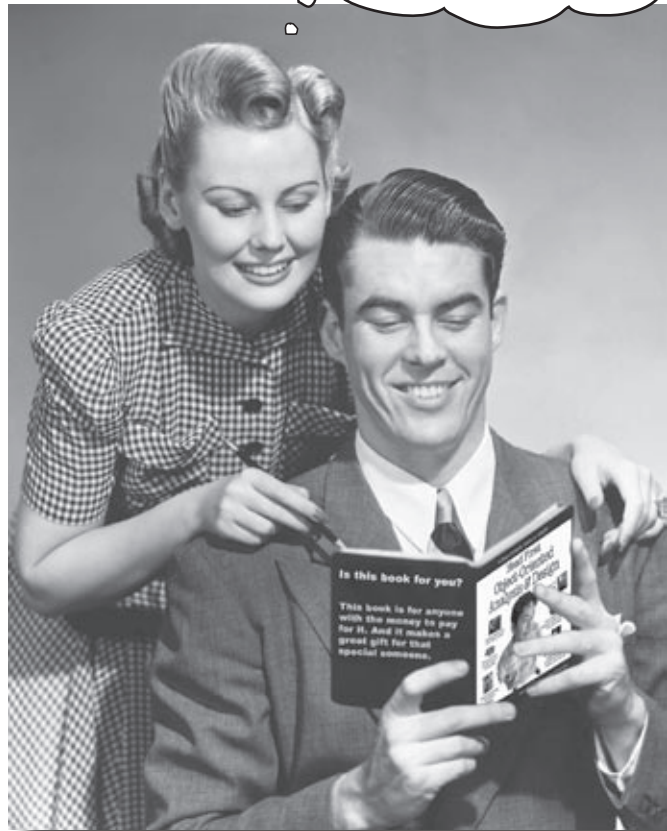
UML and class diagrams	577
Inheritance	579
Polymorphism	581
Encapsulation	582
Bullet Points	586



how to use this book

Intro

I can't believe
they put **that** in an object-
oriented analysis and design
book!



In this section, we answer the burning question:
“So why DID they put that in an OOA&D book?”

Who is this book for?

If you can answer “yes” to all of these:

- ① Do you know **Java**? (You don’t need to be a guru.) ← You’ll probably be okay if you know C# instead.
- ② Do you want to **learn, understand, remember, and apply** object-oriented analysis and design to **real world projects**, and write better software in the process?
- ③ Do you prefer **stimulating dinner party conversation** to dry, dull, academic lectures?

this book is for you.

Who should probably back away from this book?

If you can answer “yes” to any **one** of these:

- ① Are you **completely new** to Java? (You don’t need to be advanced, and even if you don’t know Java, but you know C#, you’ll probably understand almost all of the code examples. You also might be okay with just a C++ background.)
- ② Are you a kick-butt OO designer/developer looking for a **reference book**?
- ③ Are you **afraid to try something different**? Would you rather have a root canal than mix stripes with plaid? Do you believe that a technical book can’t be serious if programming concepts are anthropomorphized?

this book is not for you.



[note from marketing: this book is for anyone with a credit card.]

We know what you're thinking.

"How can *this* be a serious programming book?"

"What's with all the graphics?"

"Can I actually *learn* it this way?"

And we know what your *brain* is thinking.

Your brain craves novelty. It's always searching, scanning, *waiting* for something unusual. It was built that way, and it helps you stay alive.

So what does your brain do with all the routine, ordinary, normal things you encounter? Everything it *can* to stop them from interfering with the brain's *real* job—recording things that *matter*. It doesn't bother saving the boring things; they never make it past the "this is obviously not important" filter.

How does your brain *know* what's important? Suppose you're out for a day hike and a tiger jumps in front of you, what happens inside your head and body?

Neurons fire. Emotions crank up. *Chemicals surge.*

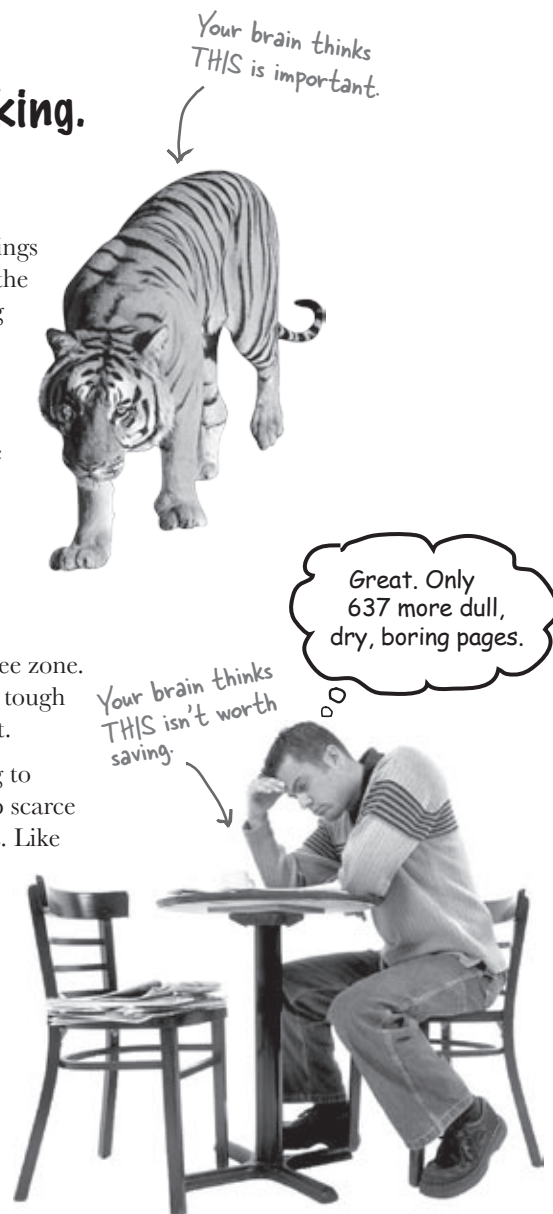
And that's how your brain knows...

This must be important! Don't forget it!

But imagine you're at home, or in a library. It's a safe, warm, tiger-free zone. You're studying. Getting ready for an exam. Or trying to learn some tough technical topic your boss thinks will take a week, ten days at the most.

Just one problem. Your brain's trying to do you a big favor. It's trying to make sure that this *obviously* non-important content doesn't clutter up scarce resources. Resources that are better spent storing the really *big* things. Like tigers. Like the danger of fire. Like how you should never again snowboard in shorts.

And there's no simple way to tell your brain, "Hey brain, thank you very much, but no matter how dull this book is, and how little I'm registering on the emotional Richter scale right now, I really *do* want you to keep this stuff around."

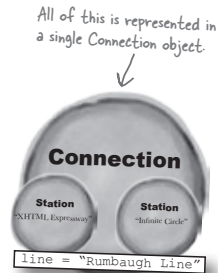


We think of a “Head First” reader as a learner.

So what does it take to *learn* something? First, you have to *get* it, then make sure you don’t *forget* it. It’s not about pushing facts into your head. Based on the latest research in cognitive science, neurobiology, and educational psychology, *learning* takes a lot more than text on a page. We know what turns your brain on.

Some of the Head First learning principles:

Make it visual. Images are far more memorable than words alone, and make learning much more effective (up to 89% improvement in recall and transfer studies). It also makes things more understandable. **Put the words within or near the graphics** they relate to, rather than on the bottom or on another page, and learners will be up to *twice* as likely to solve problems related to the content.



Use a conversational and personalized style. In recent studies,

students performed up to 40% better on post-learning tests if the content spoke directly to the reader, using a first-person, conversational style rather than taking a formal tone. Tell stories instead of lecturing. Use casual language. Don’t take yourself too seriously. Which would *you* pay more attention to: a stimulating dinner party companion, or a lecture?

It really sucks to be an abstract method. You don’t have a body.



`abstract void roam();`

No method body!
End it with a semicolon.

Get the learner to think more deeply. In other words, unless you actively flex your neurons, nothing much happens in your head. A reader has to be motivated, engaged, curious, and inspired to solve problems, draw conclusions, and generate new knowledge. And for that, you need challenges, exercises, and thought-provoking questions, and activities that involve both sides of the brain, and multiple senses.

Great software every time? I can hardly *imagine* what that would be like!



Get—and keep—the reader’s attention. We’ve all had the “I really want to learn this but I can’t stay awake past page one” experience. Your brain pays attention to things that are out of the ordinary, interesting, strange, eye-catching, unexpected.

Learning a new, tough, technical topic doesn’t have to be boring. Your brain will learn much more quickly if it’s not.

Touch their emotions. We now know that your ability to remember something is largely dependent on its emotional content. You remember what you *care* about. You remember when you *feel* something. No, we’re not talking heart-wrenching stories about a boy and his dog. We’re talking emotions like surprise, curiosity, fun, “what the...?”, and the feeling of “I Rule!” that comes when you solve a puzzle, learn something everybody else thinks is hard, or realize you know something that “I’m more technical than thou” Bob from engineering *doesn’t*.



Metacognition: thinking about thinking

If you really want to learn, and you want to learn more quickly and more deeply, pay attention to how you pay attention. Think about how you think. Learn how you learn.

Most of us did not take courses on metacognition or learning theory when we were growing up. We were *expected* to learn, but rarely *taught* to learn.

But we assume that if you're holding this book, you really want to learn object-oriented analysis and design. And you probably don't want to spend a lot of time. And since you're going to develop software, you need to *remember* what you read. And for that, you've got to *understand* it. To get the most from this book, or *any* book or learning experience, take responsibility for your brain. Your brain on *this* content.

The trick is to get your brain to see the new material you're learning as Really Important. Crucial to your well-being. As important as a tiger. Otherwise, you're in for a constant battle, with your brain doing its best to keep the new content from sticking.

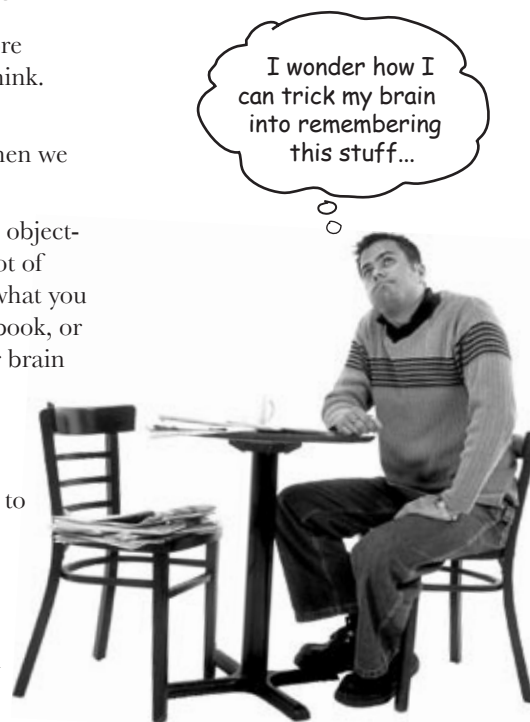
So just how **DO** you get your brain to think object-oriented analysis and design is a hungry tiger?

There's the slow, tedious way, or the faster, more effective way. The slow way is about sheer repetition. You obviously know that you *are* able to learn and remember even the duller of topics if you keep pounding the same thing into your brain. With enough repetition, your brain says, "This doesn't *feel* important to him, but he keeps looking at the same thing *over* and *over* and *over*, so I suppose it must be."

The faster way is to do **anything that increases brain activity**, especially different *types* of brain activity. The things on the previous page are a big part of the solution, and they're all things that have been proven to help your brain work in your favor. For example, studies show that putting words *within* the pictures they describe (as opposed to somewhere else in the page, like a caption or in the body text) causes your brain to try to make sense of how the words and picture relate, and this causes more neurons to fire. More neurons firing = more chances for your brain to *get* that this is something worth paying attention to, and possibly recording.

A conversational style helps because people tend to pay more attention when they perceive that they're in a conversation, since they're expected to follow along and hold up their end. The amazing thing is, your brain doesn't necessarily *care* that the "conversation" is between you and a book! On the other hand, if the writing style is formal and dry, your brain perceives it the same way you experience being lectured to while sitting in a roomful of passive attendees. No need to stay awake.

But pictures and conversational style are just the beginning.



Here's what WE did:

We used **pictures**, because your brain is tuned for visuals, not text. As far as your brain's concerned, a picture really *is* worth 1,024 words. And when text and pictures work together, we embedded the text *in* the pictures because your brain works more effectively when the text is *within* the thing the text refers to, as opposed to in a caption or buried in the text somewhere.

We used **redundancy**, saying the same thing in *different* ways and with different media types, and *multiple senses*, to increase the chance that the content gets coded into more than one area of your brain.

We used concepts and pictures in **unexpected** ways because your brain is tuned for novelty, and we used pictures and ideas with at least *some emotional content*, because your brain is tuned to pay attention to the biochemistry of emotions. That which causes you to *feel* something is more likely to be remembered, even if that feeling is nothing more than a little **humor**, **surprise**, or **interest**.

We used a personalized, **conversational style**, because your brain is tuned to pay more attention when it believes you're in a conversation than if it thinks you're passively listening to a presentation. Your brain does this even when you're *reading*.

We included more than 80 **activities**, because your brain is tuned to learn and remember more when you **do** things than when you *read* about things. And we made the exercises challenging-yet-do-able, because that's what most people prefer.

We used **multiple learning styles**, because *you* might prefer step-by-step procedures, while someone else wants to understand the big picture first, and someone else just wants to see a code example. But regardless of your own learning preference, *everyone* benefits from seeing the same content represented in multiple ways.

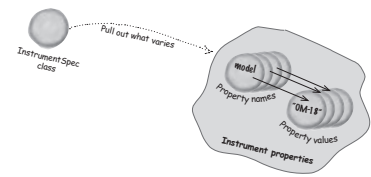
We include content for **both sides of your brain**, because the more of your brain you engage, the more likely you are to learn and remember, and the longer you can stay focused. Since working one side of the brain often means giving the other side a chance to rest, you can be more productive at learning for a longer period of time.

And we included **stories** and exercises that present **more than one point of view**, because your brain is tuned to learn more deeply when it's forced to make evaluations and judgements.

We included **challenges**, with exercises, and by asking **questions** that don't always have a straight answer, because your brain is tuned to learn and remember when it has to *work* at something. Think about it—you can't get your *body* in shape just by *watching* people at the gym. But we did our best to make sure that when you're working hard, it's on the *right* things. That **you're not spending one extra dendrite** processing a hard-to-understand example, or parsing difficult, jargon-laden, or overly terse text.

We used **people**. In stories, examples, pictures, etc., because, well, because *you're* a person. And your brain pays more attention to *people* than it does to *things*.

We used an **80/20** approach. We assume that if you're going for a PhD in software design, this won't be your only book. So we don't talk about *everything*. Just the stuff you'll actually *need*.



DO CATASTROPHE!
Chapman's Personality Quiz Show

Risk Assessment	Famous Quotes	Code Conducts	Maintenance and Repair	Software Reviews
\$100	\$100	\$100	\$100	\$100
\$200	\$200	\$200	\$200	\$200
\$300	\$300	\$300	\$300	\$300
\$400	\$400	\$400	\$400	\$400





Here's what YOU can do to bend your brain into submission

So, we did our part. The rest is up to you. These tips are a starting point; listen to your brain and figure out what works for you and what doesn't. Try new things.

Cut this out and stick it on your refrigerator. →

① **Slow down. The more you understand, the less you have to memorize.**

Don't just *read*. Stop and think. When the book asks you a question, don't just skip to the answer. Imagine that someone really *is* asking the question. The more deeply you force your brain to think, the better chance you have of learning and remembering.

② **Do the exercises. Write your own notes.**

We put them in, but if we did them for you, that would be like having someone else do your workouts for you. And don't just *look* at the exercises. **Use a pencil.** There's plenty of evidence that physical activity *while* learning can increase the learning.

③ **Read the "There are No Dumb Questions"**

That means all of them. They're not optional side-bars—***they're part of the core content!*** Don't skip them.

④ **Make this the last thing you read before bed. Or at least the last *challenging* thing.**

Part of the learning (especially the transfer to long-term memory) happens *after* you put the book down. Your brain needs time on its own, to do more processing. If you put in something new during that processing time, some of what you just learned will be lost.

⑤ **Drink water. Lots of it.**

Your brain works best in a nice bath of fluid. Dehydration (which can happen before you ever feel thirsty) decreases cognitive function.

⑥ **Talk about it. Out loud.**

Speaking activates a different part of the brain. If you're trying to understand something, or increase your chance of remembering it later, say it out loud. Better still, try to explain it out loud to someone else. You'll learn more quickly, and you might uncover ideas you hadn't known were there when you were reading about it.

⑦ **Listen to your brain.**

Pay attention to whether your brain is getting overloaded. If you find yourself starting to skim the surface or forget what you just read, it's time for a break. Once you go past a certain point, you won't learn faster by trying to shove more in, and you might even hurt the process.

⑧ **Feel something!**

Your brain needs to know that this *matters*. Get involved with the stories. Make up your own captions for the photos. Groaning over a bad joke is *still* better than feeling nothing at all.

⑨ **Design something!**

Apply what you read to something new you're designing, or rework an older project. Just do *something* to get some experience beyond the exercises and activities in this book. All you need is a problem to solve... a problem that might benefit from one or more techniques that we talk about.

Read Me

This is a learning experience, not a reference book. We deliberately stripped out everything that might get in the way of learning whatever it is we're working on at that point in the book. And the first time through, you need to begin at the beginning, because the book makes assumptions about what you've already seen and learned.

We assume you are familiar with Java.

It would take an entire book to teach you Java (in fact, that's exactly what it took: *Head First Java*). We chose to focus this book on analysis and design, so the chapters are written with the assumption that you know the basics of Java. When intermediate or advanced concepts come up, they're taught as if they might be totally new to you, though.

If you're completely new to Java, or coming to this book from a C# or C++ background, we strongly recommend you turn to the back of the book and read Appendix II before going on. That appendix has some intro material that will help you start this book off on the right foot.

We only use Java 5 when we have to.

Java 5.0 introduces a lot of new features to the Java language, ranging from generics to parameterized types to enumerated types to the **foreach** looping construct. Since many professional programmers are just moving to Java 5, we didn't want you getting hung up on new syntax while you're trying to learn about OOA&D. In most cases, we stuck with pre-Java 5 syntax. The only exception is in Chapter 1, when we needed an enumerated type—and we explained enums in that section in some detail.

If you're new to Java 5, you should have no trouble with any of the code examples. If you're already comfortable with Java 5, then you will get a few compiler warnings about unchecked and unsafe operations, due to our lack of typed collections, but you should be able to update the code for Java 5 on your own quite easily.

The activities are NOT optional.

The exercises and activities are not add-ons; they're part of the core content of the book. Some of them are to help with memory, some are for understanding, and some will help you apply what you've learned. ***Don't skip the exercises.*** The crossword puzzles are the only things you don't *have* to do, but they're good for giving your brain a chance to think about the words and terms you've been learning in a different context.

The redundancy is intentional and important.

One distinct difference in a Head First book is that we want you to *really* get it. And we want you to finish the book remembering what you’ve learned. Most reference books don’t have retention and recall as a goal, but this book is about *learning*, so you’ll see some of the same concepts come up more than once.

The examples are as lean as possible.

Our readers tell us that it’s frustrating to wade through 200 lines of an example looking for the two lines they need to understand. Most examples in this book are shown within the smallest possible context, so that the part you’re trying to learn is clear and simple. Don’t expect all of the examples to be robust, or even complete—they are written specifically for learning, and aren’t always fully-functional.

In some cases, we haven’t included all of the import statements needed, but we assume that if you’re a Java programmer, you know that **ArrayList** is in **java.util**, for example. If the imports are not part of the normal core J2SE API, we mention it. We’ve also placed all the source code on the web so you can download it. You’ll find it at <http://www.headfirstlabs.com/books/hfoo/>.

Also, for the sake of focusing on the learning side of the code, we did not put our classes into packages (in other words, they’re all in the Java default package). We don’t recommend this in the real world, and when you download the code examples from this book, you’ll find that all classes *are* in packages.

The ‘Brain Power’ exercises don’t have answers.

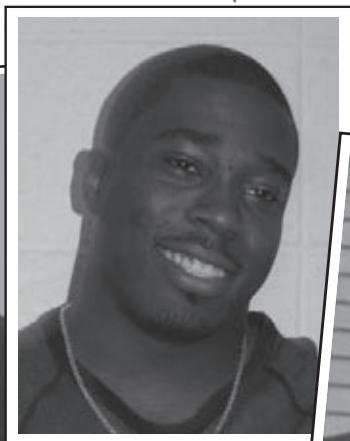
For some of them, there is no right answer, and for others, part of the learning experience of the Brain Power activities is for you to decide if and when your answers are right. In some of the Brain Power exercises you will find hints to point you in the right direction.

The Technical Team

Ara Yapejian



Hannibal Scipio



Chris Austin



Technical Reviewers:

Huge thanks to our amazing trio of technical reviewers. These guys caught mistakes that we missed, let us know when we were moving too fast (or too slow), and even let us know when our jokes sucked. Several times, they turned chapters around in a matter of hours... we're not sure if that means they're really helpful, or need to get away from software development a little more. **Hannibal** in particular made our week when he let us know that the big OOA&D arrow in Chapter 10 was "Hot!" Thanks guys, this book wouldn't be nearly as solid without your hard work.

Kathy Sierra and Bert Bates:

We continue to be amazed at the insight and expertise that **Bert Bates** has about cliffs, and that **Kathy Sierra** has about dog doors. If that doesn't make much sense, don't be surprised—everything you know about almost everything gets turned on its head when you meet this pair, and yet we all came out much for the better because of their help.

Bert and Kathy did a ton of review at the eleventh hour, and we're thankful they did. Their help and guidance continues to be the heart of Head First.



Kathy Sierra



Bert Bates

Acknowledgements

My co-authors:

Because I'm doing the typing, I get to step out of "we" mode for a moment and say thanks to my co-authors, **Dave West** and **Gary Pollice**. Neither of these guys knew what they were signing up for when they came on board, but I've never been so impressed by a couple of guys willing to explain, defend, and even change their opinions and knowledge about software design, requirements and analysis, and lift shafts. They were simply incredible, writing up until the very last day, and even got me to relax and laugh until I cried on several occasions.

Our editor:



↑ Mary O'Brien

This book wouldn't be in your hands if not for **Mary O'Brien**. I think it's fair to say she fought more battles and paved the way for us to work without interruption more times than any of us really are aware of. Most importantly, she made this the single most enjoyable project we've worked on in our careers. Frankly, she kicked our asses a number of times, and it made all the difference. She really doesn't realize how much of an effect she has on the people she works with, because we don't tell her enough how much we respect her and value her opinions. So there, now you know, Mary. If we could put your name on the cover, we would (oh, wait... we did!).

The O'Reilly team:

These books are a team effort, never more so than on this one. **Mike Hendrickson** and **Laurie Petrycki** oversaw this project at various times, and took heated phone calls more than once. **Sanders Kleinfeld** cut his Head First teeth on this project, and managed to come out alive; better yet, he did a great job, improving the book, and we all are excited that this is just the first of many Head First books he'll be working on. **Mike Loukides** found Bert and Kathy way back when, and **Tim O'Reilly** had the foresight to turn their crazy idea into a series. As always, **Kyle Hart** is instrumental in getting these books "out there", and **Edie Freedman's** beautiful cover design continues to amaze us all.

A particularly special thanks goes out to **Louise Barr**, the Head First Design Editor. Lou pulled several 12- and 14-hour days to help us with graphics in this book, and put together the amazing Objectville Subway Map in Chapter 10. Lou, your work has improved the learning quality of this book, and we can't thank you enough for your contributions.

→ Lou Barr



Special thanks

Near the completion of this book, **Laura Baldwin**, the CFO of O'Reilly, encountered some personal tragedy. It's hard to know what to say in these situations, especially because Laura has really become the backbone of O'Reilly in many ways. Laura, we are thinking and praying for you and your family, and we wish you all the very, very best in the days to come. We know you'd want nothing more than to see everyone at O'Reilly working harder than ever while you're away.

This book is certainly a testament to the people at O'Reilly continuing to deliver, and in many of our conversations, your name came up as someone we wanted to support, and not let down in any way. Your effect on this company is extraordinary, and O'Reilly and the Head First series will all be much better for the day you can return to us in full swing.

1 well-designed apps rock

Great Software Begins Here

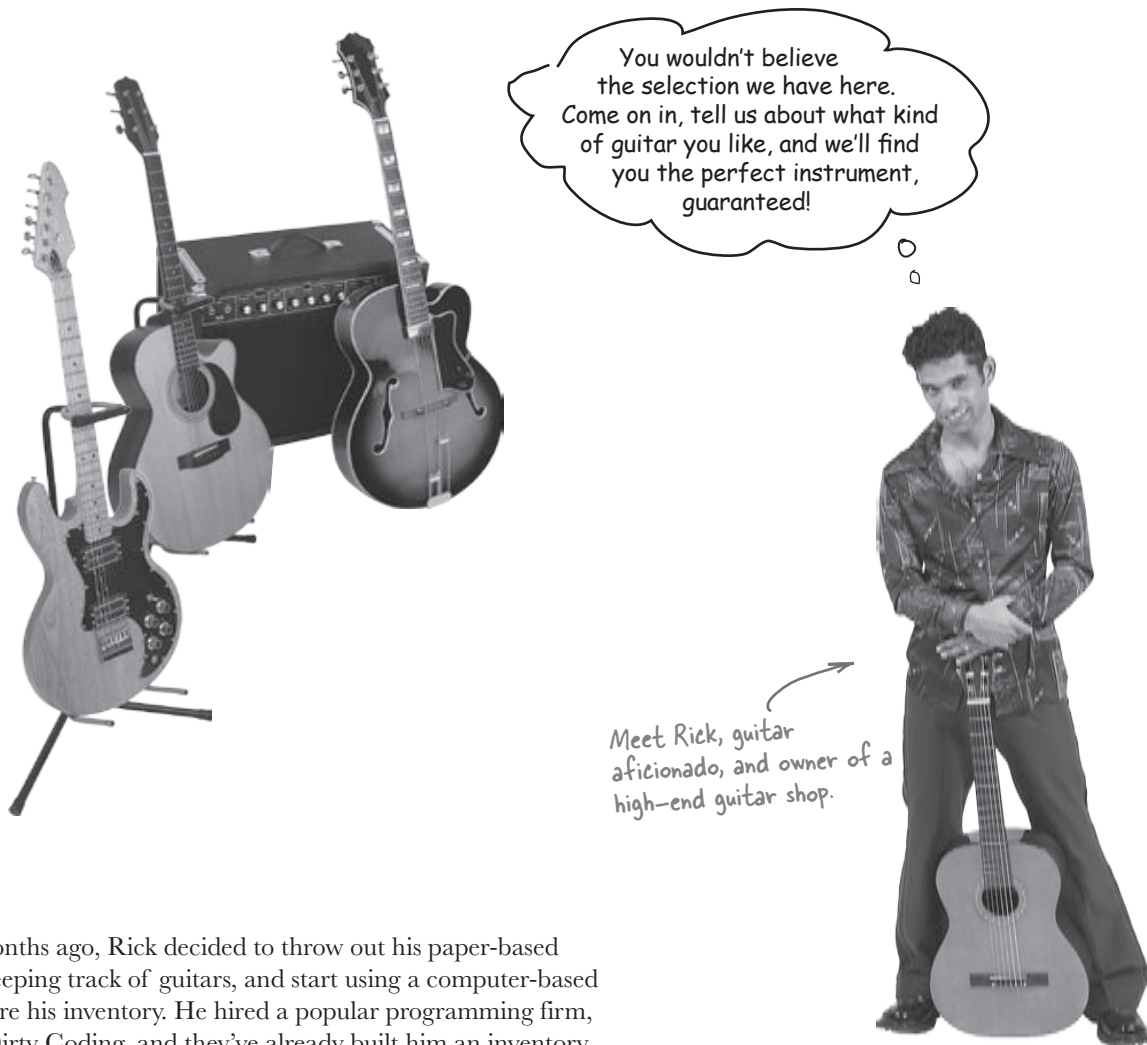
I can hardly get over it,
Sue, but since I started using
OOA&D, I'm just a new man... a
new man, I'll tell you!



So how do you *really* write great software? It's never easy trying to figure out **where to start**. Does the application actually **do what it's supposed to**? And what about things like duplicate code—that can't be good, can it? It's usually pretty hard to know **what you should work on first**, and still make sure you don't screw everything else up in the process. No worries here, though. By the time you're done with this chapter, you'll **know how to write great software**, and be well on your way to improving the way you develop applications forever. Finally, you'll understand why **OOA&D** is a four-letter word that your mother actually *wants* you to know about.

Rock and roll is forever!

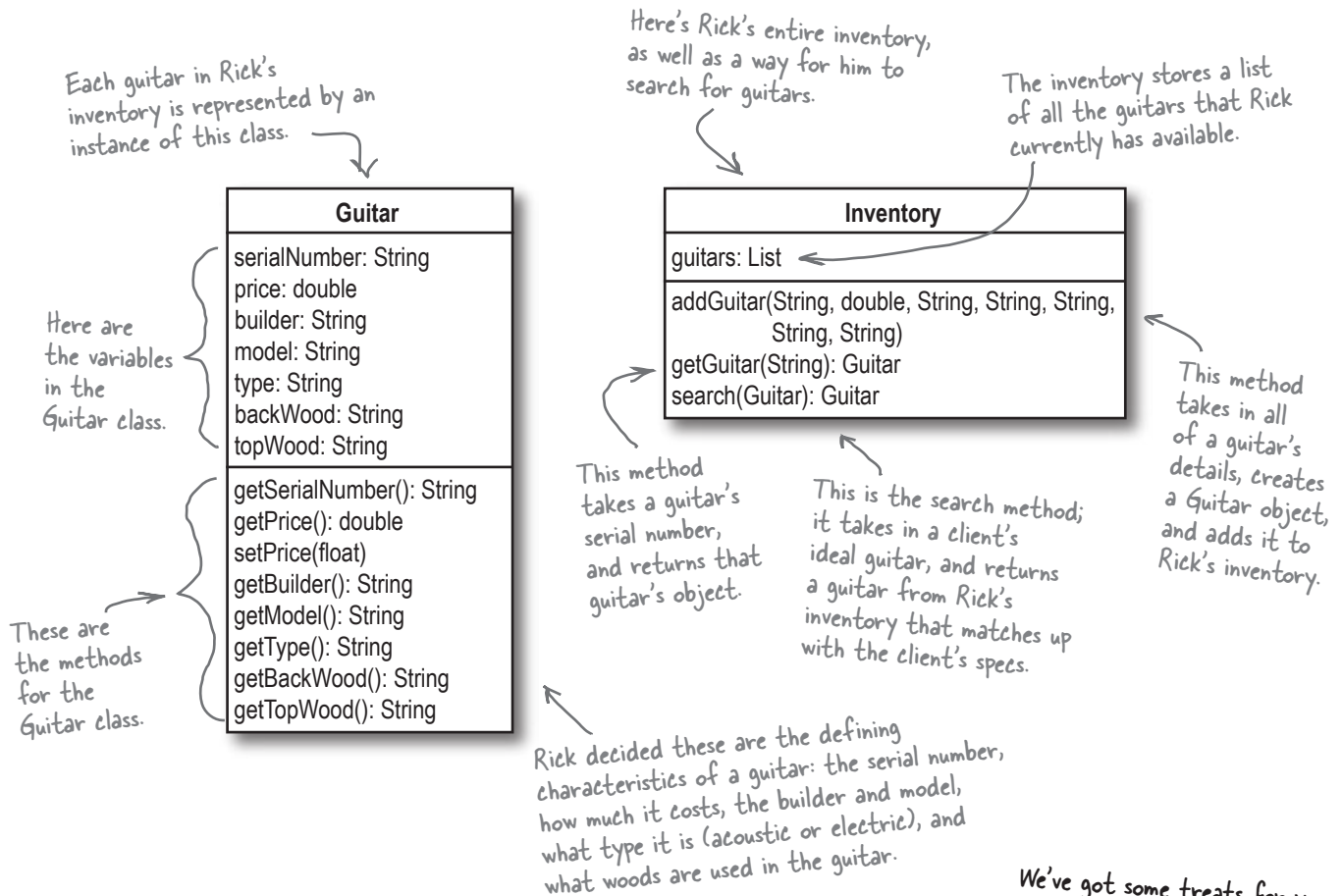
There's nothing better than the sound of a killer guitar in the hands of a great player, and Rick's Guitars specializes in finding the perfect instrument for his discerning customers.



Just a few months ago, Rick decided to throw out his paper-based system for keeping track of guitars, and start using a computer-based system to store his inventory. He hired a popular programming firm, Down and Dirty Coding, and they've already built him an inventory management app. He's even had the firm build him a new search tool to help him match up a customer to their dream instrument.

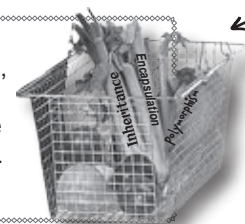
Rick's shiny new application...

Here's the application that the programming firm built for Rick... they've put together a system to completely replace all of Rick's handwritten notes, and help him match his customers with the perfect guitar. Here's the UML class diagram they gave Rick to show him what they did:



New to Objectville?

If you're new to object oriented programming, haven't heard of UML before, or aren't sure about the diagram above, it's OK! We've prepared a special "Welcome to Objectville" care package for you to get you started. Flip to the back of the book, and read Appendix II—we promise you'll be glad you did. Then come back here, and this will all make a lot more sense.



We've got some treats for you in Appendix II. Check it out before going on if you're new to UML or OO programming.

Here what the code for Guitar.java looks like

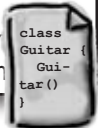
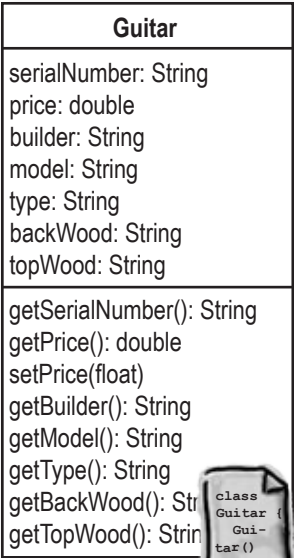
You've seen the class diagram for Rick's application on the last page; now let's look at what the actual code for **Guitar.java** and **Inventory.java** look like.

```
public class Guitar {  
  
    private String serialNumber, builder, model, type, backWood, topWood;  
    private double price;  
  
    public Guitar(String serialNumber, double price,  
                  String builder, String model, String type,  
                  String backWood, String topWood) {  
        this.serialNumber = serialNumber;  
        this.price = price;  
        this.builder = builder;  
        this.model = model;  
        this.type = type;  
        this.backWood = backWood;  
        this.topWood = topWood;  
    }  
  
    public String getSerialNumber() {  
        return serialNumber;  
    }  
  
    public double getPrice() {  
        return price;  
    }  
    public void setPrice(double newPrice) {  
        this.price = newPrice;  
    }  
    public String getBuilder() {  
        return builder;  
    }  
    public String getModel() {  
        return model;  
    }  
    public String getType() {  
        return type;  
    }  
    public String getBackWood() {  
        return backWood;  
    }  
    public String getTopWood() {  
        return topWood;  
    }  
}
```

These are all the properties we saw from the class diagram for the Guitar class.

UML class diagrams don't show constructors; the Guitar constructor does just what you'd expect, though: sets all the initial properties for a new Guitar.

You can see how the class diagram matches up with the methods in the Guitar class's code.



Guitar.java

And Inventory.java...

```
public class Inventory {
    private List guitars;
```

← Remember, we've stripped out the import statements to save some space.

```
    public Inventory() {
        guitars = new LinkedList();
    }
```

```
    public void addGuitar(String serialNumber, double price,
                          String builder, String model,
                          String type, String backWood, String topWood) {
        Guitar guitar = new Guitar(serialNumber, price, builder,
                                   model, type, backWood, topWood);
        guitars.add(guitar);
    }
```

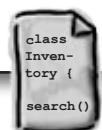
← addGuitar() takes in all the properties required to create a new Guitar instance, creates one, and adds it to the inventory.

```
    public Guitar getGuitar(String serialNumber) {
        for (Iterator i = guitars.iterator(); i.hasNext(); ) {
            Guitar guitar = (Guitar)i.next();
            if (guitar.getSerialNumber().equals(serialNumber)) {
                return guitar;
            }
        }
        return null;
    }
```

```
    public Guitar search(Guitar searchGuitar) {
        for (Iterator i = guitars.iterator(); i.hasNext(); ) {
            Guitar guitar = (Guitar)i.next();
            // Ignore serial number since that's unique
            // Ignore price since that's unique
            String builder = searchGuitar.getBuilder();
            if ((builder != null) && (!builder.equals("")) &&
                (!builder.equals(guitar.getBuilder())))
                continue;
            String model = searchGuitar.getModel();
            if ((model != null) && (!model.equals("")) &&
                (!model.equals(guitar.getModel())))
                continue;
            String type = searchGuitar.getType();
            if ((type != null) && (!searchGuitar.equals("")) &&
                (!type.equals(guitar.getType())))
                continue;
            String backWood = searchGuitar.getBackWood();
            if ((backWood != null) && (!backWood.equals("")) &&
                (!backWood.equals(guitar.getBackWood())))
                continue;
            String topWood = searchGuitar.getTopWood();
            if ((topWood != null) && (!topWood.equals("")) &&
                (!topWood.equals(guitar.getTopWood())))
                continue;
        }
        return null;
    }
}
```

← This method is a bit of a mess... it compares each property of the Guitar object it's passed in to each Guitar object in Rick's inventory.

Inventory
guitars: List
addGuitar(String, double, String, String, String, String, String)
getGuitar(String): Guitar
search(Guitar): Guitar



Inventory.java

But then Rick started losing customers...

It seems like no matter who the customer is and what they like, Rick's new search program almost always comes up empty when it looks for good guitar matches. But Rick knows he has guitars that these customers would like... so what's going on?

FindGuitarTester.java simulates a typical day for Rick now... a customer comes in, tells him what they like, and he runs a search on his inventory.

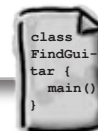
```
public class FindGuitarTester {

    public static void main(String[] args) {
        // Set up Rick's guitar inventory
        Inventory inventory = new Inventory();
        initializeInventory(inventory);

        Guitar whatErinLikes = new Guitar("", 0, "fender", "Stratocaster",
                                           "electric", "Alder", "Alder");
        Guitar guitar = inventory.search(whatErinLikes);
        if (guitar != null) {
            System.out.println("Erin, you might like this " +
                               guitar.getBuilder() + " " + guitar.getModel() + " " +
                               guitar.getType() + " guitar:\n    " +
                               guitar.getBackWood() + " back and sides,\n    " +
                               guitar.getTopWood() + " top.\nYou can have it for only $" +
                               guitar.getPrice() + "!");
        } else {
            System.out.println("Sorry, Erin, we have nothing for you.");
        }
    }

    private static void initializeInventory(Inventory inventory) {
        // Add guitars to the inventory...
    }
}
```

Erin is looking for a Fender "Strat" guitar, made of Alder.



FindGuitarTester.java

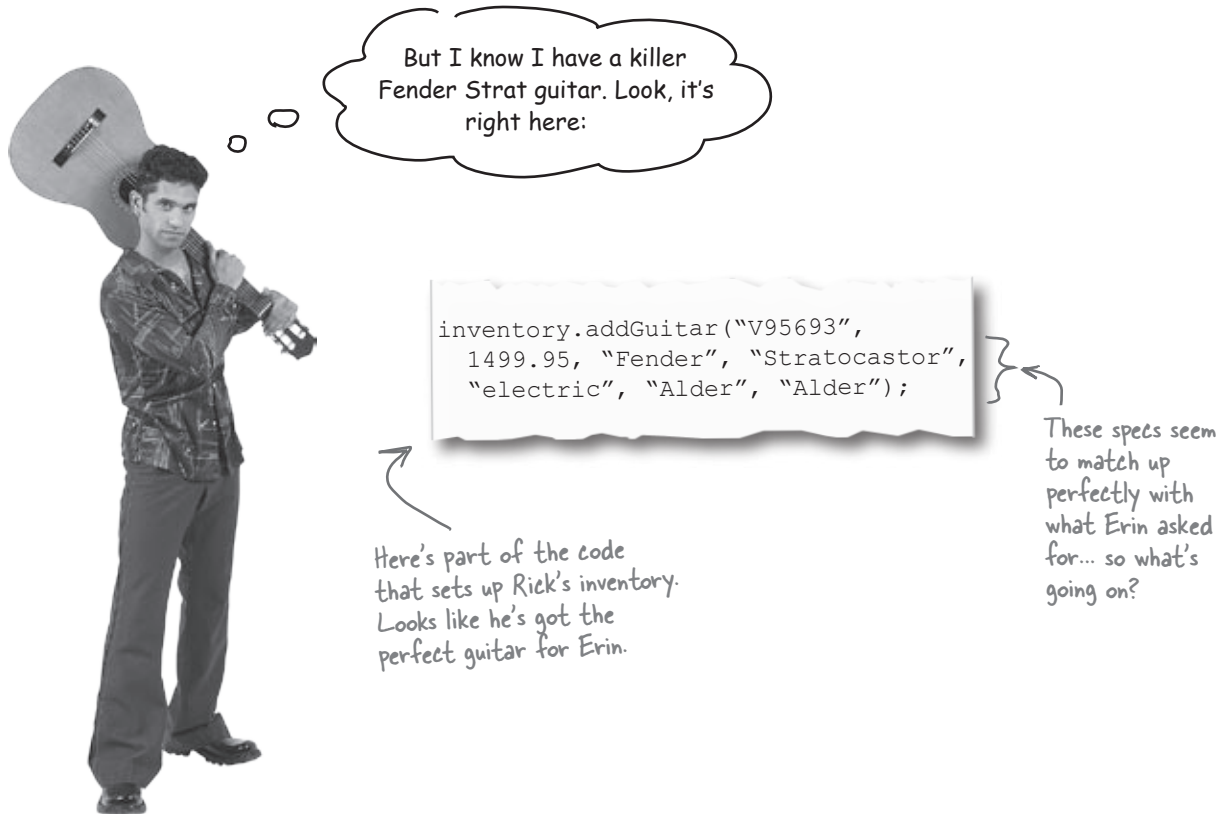
File Edit Window Help C7#5

```
%java FindGuitarTester
Sorry, Erin, we have nothing for you.
```

I'm sorry, Rick, I guess I'll just go to that other store across town.

Here's what happens when Erin comes into Rick's store, and Rick tries to find her a guitar.



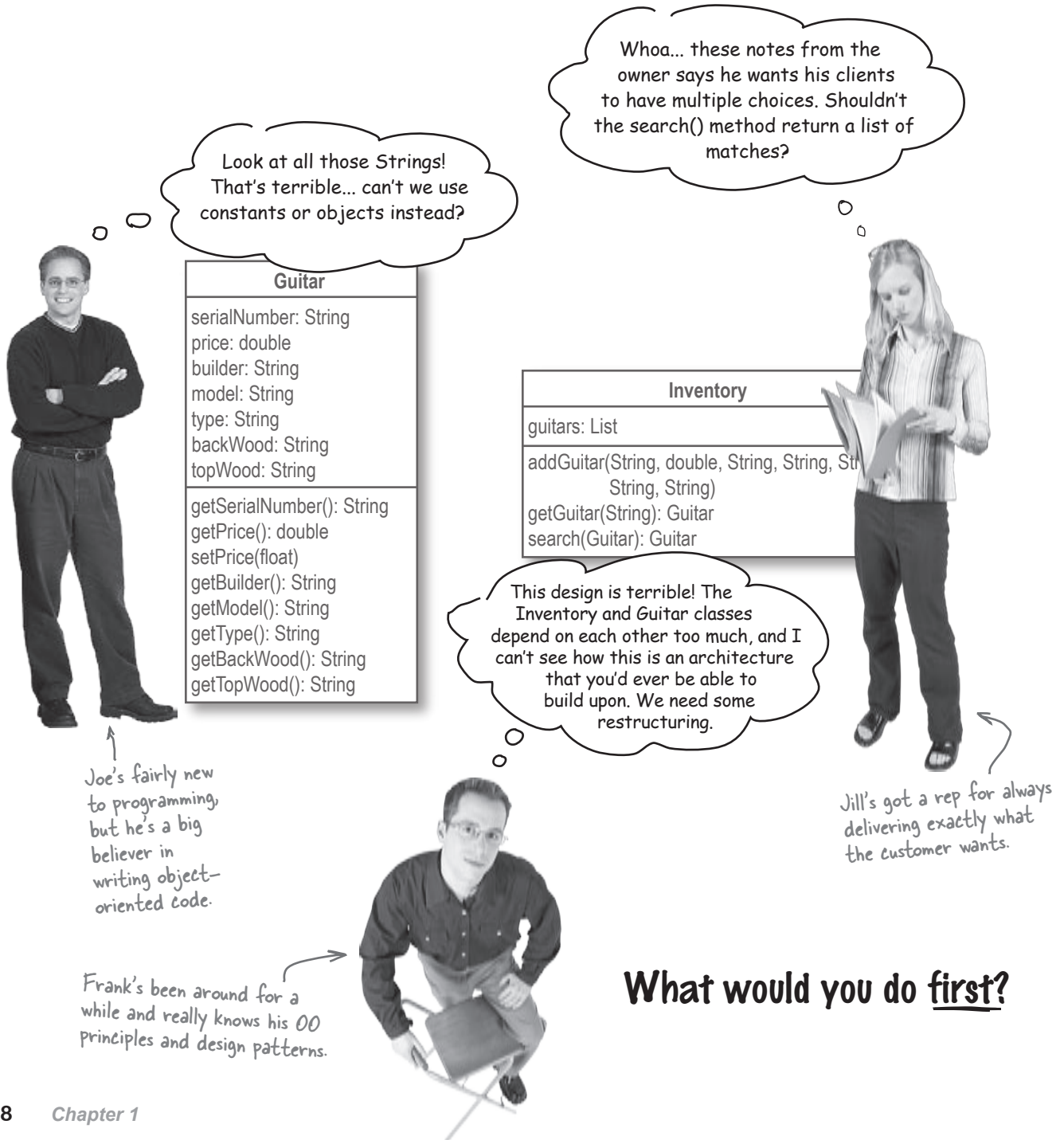


How would you redesign Rick's app?

Look over the last three pages, showing the code for Rick's app, and the results of running a search. What problems do you see? What would you change? Write down the **FIRST** thing you'd do to improve Rick's app in the blanks below.

What's the FIRST thing you'd change?

It's obvious that Rick's app has problems, but it's not so obvious what we should work on first. And it looks like there's no shortage of opinion:



What would you do first?

How am I supposed to know where to start? I feel like every time I get a new project to work on, everyone's got a different opinion about what to do first. Sometimes I get it right, and sometimes I end up reworking the whole app because I started in the wrong place. **I just want to write great software!** So what should I do first in Rick's app?



How do you
write great
software,
every time?



Wait a second... I hate to butt in, but what does "great software" mean? That's sort of a vague term to be throwing around, isn't it?

Good question... and there are lots of different answers:

The customer-friendly programmer says:

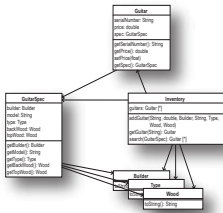
"Great software always does what the customer wants it to. So even if customers think of new ways to use the software, it doesn't break or give them unexpected results."



This approach is all about making sure the customer is happy with what their app does.

The object-oriented programmer says:

"Great software is code that is object-oriented. So there's not a bunch of duplicate code, and each object pretty much controls its own behavior. It's also easy to extend because your design is really solid and flexible."

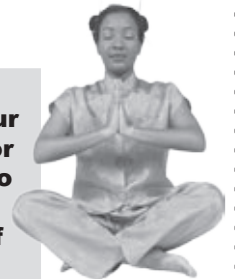


Good OO programmers are always looking for ways to make their code more flexible.

Not sure about what all that means? It's OK... you'll learn about all these things in the upcoming chapters.

The design-guru programmer says:

"Great software is when you use tried-and-true design patterns and principles. You've kept your objects loosely coupled, and your code open for extension but closed for modification. That also helps make the code more reusable, so you don't have to rework everything to use parts of your application over and over again."





What do **you** think “great software” means?

You’ve seen what several different types of programmers think great software is... so who is right? Or do you have your own definition of what makes an application great? It’s your turn to write down what you think makes for great software:

Write your name here...

...and write what you think great software means here.

_____ **says:**

“

_____”

Great software is... more than just one thing

It's going to take more than just a simple definition to figure out exactly what "great software" means. In fact, *all* of the different programmers on page 10 talked about a *part* of what makes software great.

First, great software must satisfy the customer. The software must do what the customer wants it to do.



Win your customers over

Customers will think your software is great when it does what it's supposed to do.

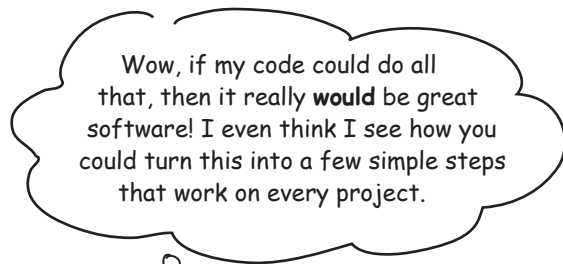
Building software that works right is great, but what about when it's time to add to your code, or reuse it in another application? It's not enough to just have software that works like the customer wants it to; your software better be able to stand the test of time.

Second, great software is well-designed, well-coded, and easy to maintain, reuse, and extend.



Make your code as smart as you are.

You (and your co-workers) will think your software is great when it's easy to maintain, reuse, and extend.



Wow, if my code could do all that, then it really **would** be great software! I even think I see how you could turn this into a few simple steps that work on every project.



Great software in 3 easy steps

It may not seem easy now, but we'll show you how OOA&D and some basic principles can change your software forever.

1. Make sure your software does what the customer wants it to do.

← This step focuses on the customer. Make sure the app does what it's supposed to do FIRST. This is where getting good requirements and doing some analysis comes in.

2. Apply basic OO principles to add flexibility.

Once your software works, you can look for any duplicate code that might have slipped in, and make sure you're using good OO programming techniques.

3. Strive for a maintainable, reusable design.

Got a good object-oriented app that does what it should? It's time to apply patterns and principles to make sure your software is ready to use for years to come.

Remember Rick? Remember his lost customers?

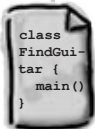
Let's put our ideas about how to write great software to the test and see if they hold up in the real world. Rick's got a search tool that isn't working, and it's your job to fix the application, and turn it into something great. Let's look back at the app and see what's going on:

Here's our test program that reveals a problem with the search tool.

```
public class FindGuitarTester {  
  
    public static void main(String[] args) {  
        // Set up Rick's guitar inventory  
        Inventory inventory = new Inventory();  
        initializeInventory(inventory);  
  
        Guitar whatErinLikes = new Guitar("", 0, "fender", "Stratocaster",  
                                           "electric", "Alder", "Alder");  
  
        Guitar guitar = inventory.search(whatErinLikes);  
        if (guitar != null) {
```

Rick's app should match Erin's preferences here...

...to this guitar in Rick's inventory.



FindGuitarTester.java

```
inventory.addGuitar("V95693",  
                    1499.95, "Fender", "Stratocaster",  
                    "electric", "Alder", "Alder");
```

So let's apply our 3 steps:

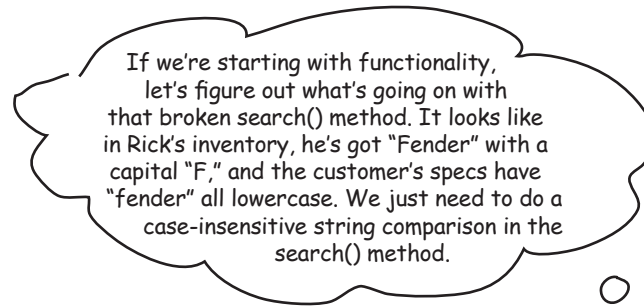
1. Make sure your software does what the customer wants it to do.

Remember, we need to start out by making sure the app actually does what Rick wants... and it's definitely not doing that right now.

2. Apply basic OO principles to add flexibility.

Don't worry too much about trying to apply patterns or other OO techniques to your app at this point... just get it to where it's working like it should.

3. Strive for a maintainable, reusable design.



Let's get a little help from some of our programmer buddies.



Frank: Sure, that would fix the problem Rick's having now, but I think there's probably a better way to make this work than just calling `toLowerCase()` on a bunch of strings all over the place.

Joe: Yeah, I was thinking the same thing. I mean, all that string comparison seems like a bad idea. Couldn't we use constants or maybe some enumerated types for the builders and woods?

Jill: You guys are thinking *way* too far ahead. Step 1 was supposed to be fixing the app so it does what the customer wants it to do. I thought we weren't supposed to worry about design yet.

Frank: Well, yeah, I get that we're supposed to focus on the customer. But we can at least be smart about *how* we fix things, right? I mean, why create problems we'll have to come back and fix later on if we can avoid them from the start?

Jill: Hmmm... I guess that does make sense. We don't want our solution to this problem creating new design problems for us down the road. But we're still not going to mess with the other parts of the application, right?

Frank: Right. We can just remove all those strings, and the string comparisons, to avoid this whole case-matching thing.

Joe: Exactly. If we go with enumerated types, we can ensure that only valid values for the builder, woods, and type of guitar are accepted. That'll make sure that Rick's clients actually get to look at guitars that match their preferences.

Jill: And we've actually done a little bit of design at the same time... very cool! Let's put this into action.

Don't create problems to solve problems.

Ditching String comparisons

The first improvement we can make to Rick's guitar search tool is getting rid of all those annoying **String** comparisons. And even though you could use a function like `toLowerCase()` to avoid problems with uppercase and lowercase letters, let's avoid **String** comparisons altogether:

Each enum takes the place of one of the guitar properties that is standard across all guitars.

We can refer to these as `Wood.SITKA`, or `Builder.GIBSON`, and avoid all those string comparisons completely.

One of the big advantages of using enums is that it limits the possible values you can supply to a method... no more misspellings or case issues.

These are all Java enums, enumerated types that function sort of like constants.

```

public enum Type {
    ACOUSTIC, ELECTRIC;

    public String toString() {
        switch(this) {
            case ACOUSTIC: return "acoustic";
            case ELECTRIC: return "electric";
        }
    }
}

public enum Builder {
    FENDER, MARTIN, GIBSON, COLLINGS,
    OLSON, RYAN, PRS, ANY;

    public String toString() {
        switch(this) {
            case FENDER: return "Fender";
            case MARTIN: return "Martin";
            case GIBSON: return "Gibson";
            case COLLINGS: return "Collings";
            case OLSON: return "Olson";
            case RYAN: return "Ryan";
            case PRS: return "PRS";
            case ANY: return "Any";
        }
    }
}

public enum Wood {
    INDIAN_ROSEWOOD, BRAZILIAN_ROSEWOOD, MAHOGANY,
    MAPLE, COCOBOLO, CEDAR, ADIRONDACK, ALDER, SITKA;

    public String toString() {
        switch(this) {
            case INDIAN_ROSEWOOD: return "Indian Rosewood";
            case BRAZILIAN_ROSEWOOD: return "Brazilian Rosewood";
            case MAHOGANY: return "Mahogany";
            case MAPLE: return "Maple";
            case COCOBOLO: return "Cocobolo";
            case CEDAR: return "Cedar";
            case ADIRONDACK: return "Adirondack";
            case ALDER: return "Alder";
            case SITKA: return "Sitka";
        }
    }
}

```

Type.java

Builder.java

Wood.java

there are no Dumb Questions

Q: I've never seen an enum before. What is that, exactly?

A: Enums are *enumerated types*. They're available in C, C++, Java version 5.0 and up, and will even be a part of Perl 6.

Enumerated types let you define a type name, like `Wood`, and then a set of values that are allowed for that type (like `COCOBOLO`, `SITKA`, and `MAHOGANY`). Then, you refer to a specific value like this: `Wood.COCOBOLO`.

Q: And why are enumerated types so helpful here?

```
public class FindGuitarTester {
```

```
    public static void main(String[] args) {
        // Set up Rick's guitar inventory
        Inventory inventory = new Inventory();
        initializeInventory(inventory);
```

We can replace all those String preferences with the new enumerated type values.

```
        Guitar whatErinLikes = new Guitar("", 0, Builder.FENDER,
            "Stratocaster", Type.ELECTRIC, Wood.ALDER, Wood.ALDER);
        Guitar guitar = inventory.search(whatErinLikes);
        if (guitar != null) {
```

The only String left is for the model, since there really isn't a limited set of these like there is with builders and wood.

```
class
FindGui-
tar {
    main()
}
```

FindGuitarTester.java

```
    public Guitar search(Guitar searchGuitar) {
        for (Iterator i = guitars.iterator(); i.hasNext(); ) {
            Guitar guitar = (Guitar)i.next();
            // Ignore serial number since that's unique
            // Ignore price since that's unique
            if (searchGuitar.getBuilder() != guitar.getBuilder())
                continue;
            String model = searchGuitar.getModel().toLowerCase();
            if ((model != null) && (!model.equals("")) &&
                (!model.equals(guitar.getModel().toLowerCase())))
                continue;
            if (searchGuitar.getType() != guitar.getType())
                continue;
            if (searchGuitar.getBackWood() != guitar.getBackWood())
                continue;
            if (searchGuitar.getTopWood() != guitar.getTopWood())
                continue;
            return guitar;
        }
        return null;
    }
```

It looks like nothing has changed, but with enums, we don't have to worry about these comparisons getting screwed up by misspellings or case issues.

The only property that we need to worry about case on is the model, since that's still a String.

```
class
Inven-
tory {
    search()
}
```

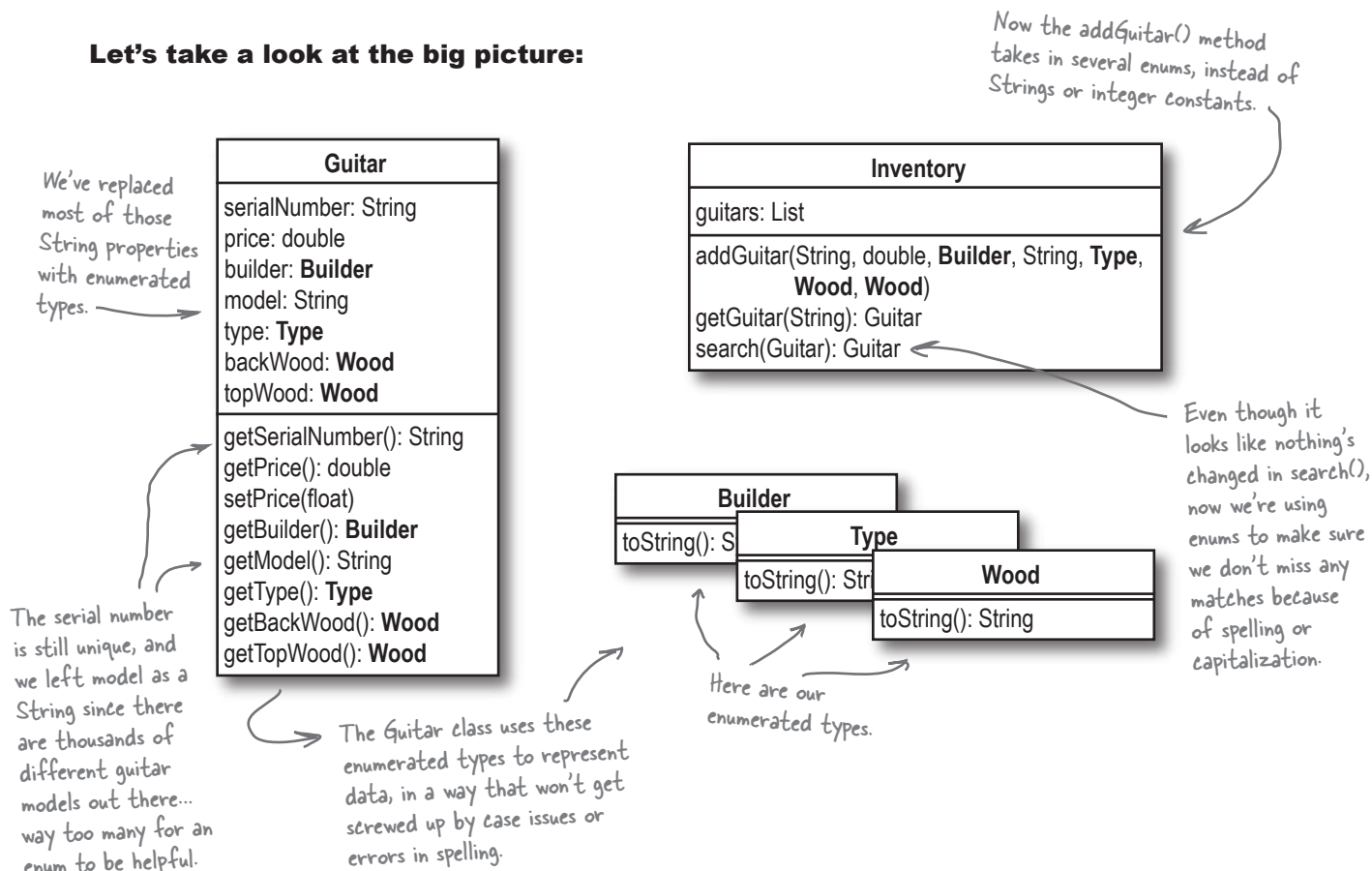
Inventory.java

A: The cool thing about enums is that methods or classes that use them are protected from any values not defined in the enum. So you can't misspell or mistype an enum without getting a compiler error. It's a great way to get not only type safety, but value safety; you can avoid getting bad data for anything that has a standard range or set of legal values.

Q: I'm using an older version of Java. Am I stuck?

A: No, not at all. Visit the Head First Labs web site at <http://www.headfirstlabs.com>, where we've posted a version of Rick's Guitars that doesn't use enums, and will work with older JDKs.

Let's take a look at the big picture:



So what have we really done here?

We've gotten a lot closer to completing step 1 in building great software. Rick's problem with searches coming up empty when he's got a matching guitar in his inventory is a thing of the past.

Even better, we've made Rick's application *less fragile* along the way. It's not going to break so easily now, because we've added both type safety and value safety with these enums. That means less problems for Rick, and less maintenance for us.

Code that is not fragile is generally referred to as robust code.

1. Make sure your software does what the customer wants it to do.



Sharpen your pencil

Apply Step 1 to your own project.

It's time to see how you can satisfy your own customers. In the blank below, write a short description of the current project you're working on (you can also use a project you finished recently):

Now, write down the first thing you did when you started working on this project. Did it have anything to do with making sure your code did what the customer wanted it to?

If you started out focusing on something other than the customer, think about how you might have approached things differently if you knew about the 3 steps to building great software. What would have been different? Do you think your application would be any better or worse than it is right now?

there are no Dumb Questions

Q: So it's OK to do a *little* design when I'm working on Step 1, right?

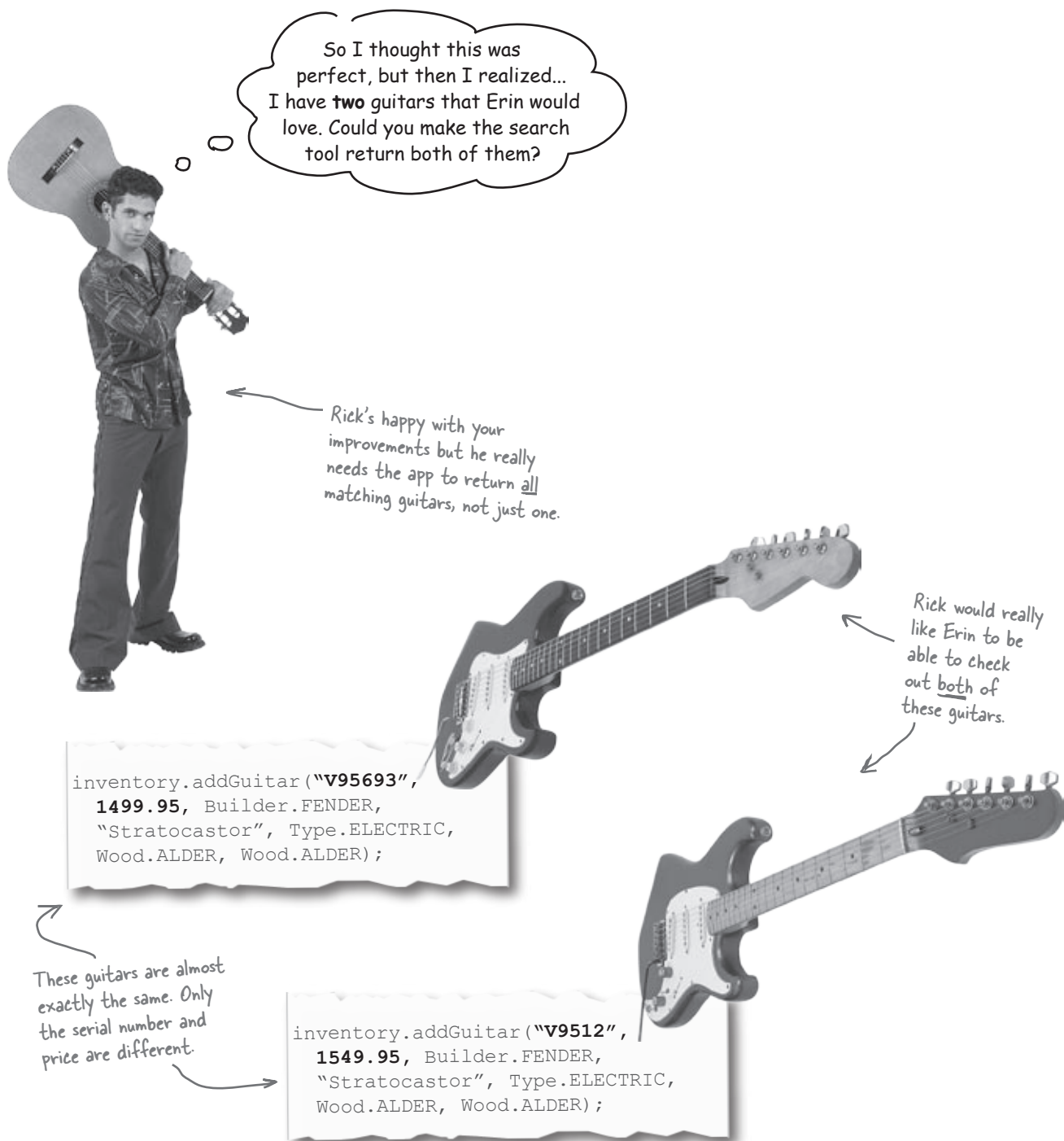
A: Yeah, as long as your focus is still on the customer's needs. You want the basic features of your application in place *before* you start making big design changes. But while you're working on functionality, you can certainly use good OO principles and techniques to make sure your application is well designed from the start.

Q: That diagram over on page 18 is a class diagram right? Or is it class diagrams, since it's more than one class?

A: It is a class diagram, and a single diagram can have multiple classes in it. In fact, class diagrams can show a lot more detail than you've seen so far, and we'll be adding to them in the next several chapters.

Q: So we're ready to move on to Step 2, and start applying OO principles, right?

A: Not quite... there's one more thing Rick would like us to help him with before we're ready to start analyzing our code for places we might be able to improve it. Remember, our first job is to please the customer, and *then* we really focus on improving our OO design.



Rick's customers want choices!

Rick's come up with a new requirement for his app: he wants his search tool to return *all* the guitars that match his client's specs, not just the first one in his inventory.

Inventory
guitars: List
addGuitar(String, double, Builder, String, Type, Wood, Wood)
getGuitar(String): Guitar
search(Guitar): List

We want search() to be able to return multiple Guitar objects if Rick has more than one guitar that matches his client's specs.



Code Magnets

Let's continue with Step 1, and make sure we've got the app working right. Below is the code for the search() method in Rick's inventory tool, but it's up to you to fill in the missing pieces. Use the code magnets at the bottom of the page to return all the matching guitars from Rick's inventory.

```
public _____ search(Guitar searchGuitar) {
    _____ = new _____();
    for (Iterator i = guitars.iterator(); i.hasNext(); ) {
        Guitar guitar = (Guitar)i.next();
        // Ignore serial number since that's unique
        // Ignore price since that's unique
        if (searchGuitar.getBuilder() != guitar.getBuilder())
            continue;
        String model = searchGuitar.getModel();
        if ((model != null) && (!model.equals("")) &&
            (!model.equals(guitar.getModel())))
            continue;
        if (searchGuitar.getType() != guitar.getType())
            continue;
        if (searchGuitar.getBackWood() != guitar.getBackWood())
            continue;
        if (searchGuitar.getTopWood() != guitar.getTopWood())
            continue;
        _____ . _____ (_____);
    }
    return _____;
}
```





Code Magnets

Let's keep on with Step 1, and make sure we've got the app working right. Below is the code for the search() method in Rick's inventory tool, but it's up to you to fill in the missing pieces. Use the code magnets at the bottom of the page to return all the matching guitars from Rick's inventory.

```
public List search(Guitar searchGuitar) {
    List matchingGuitars = new LinkedList();
    for (Iterator i = guitars.iterator(); i.hasNext(); ) {
        Guitar guitar = (Guitar)i.next();
        // Ignore serial number since that's unique
        // Ignore price since that's unique
        if (searchGuitar.getBuilder() != guitar.getBuilder())
            continue;
        String model = searchGuitar.getModel();
        if ((model != null) && (!model.equals("")) &&
            (!model.equals(guitar.getModel())))
            continue;
        if (searchGuitar.getType() != guitar.getType())
            continue;
        if (searchGuitar.getBackWood() != guitar.getBackWood())
            continue;
        if (searchGuitar.getTopWood() != guitar.getTopWood())
            continue;
        matchingGuitars.add(guitar);
    }
    return matchingGuitars;
}
```

Matching guitars
get added to the
list of options
for Rick's client.

You actually could
have used either a
LinkedList or an
ArrayList here...
both choices are OK.

Leftover magnets.

there are no Dumb Questions

Q: So I'm not done with the first step until the application works like my customer wants it to?

A: Exactly. You want to make sure that the application works like it should before you dive into applying design patterns or trying to do any real restructuring of how the application is put together.

Q: And why is it so important to finish Step 1 before going on to Step 2?

A: You're going to make lots of changes to your software when you're getting it to work right. Trying to do too much design before you've at least got the basic functionality down can end up being a waste, because a lot of the design will change as you're adding new pieces of functionality to your classes and methods.

Q: You seem sort of hung up on this "Step 1" and "Step 2" business. What if I don't code my apps that way?

A: There's nothing that says you have to follow these steps exactly, but they do provide an easy path to follow to make sure your software does what it's supposed to, and is well-designed and easy to reuse. If you've got something similar that accomplishes the same goals, that's great!

Test drive

We've talked a lot about getting the right requirements from the customer, but now we need to make sure we've actually got those requirements handled by our code. Let's test things out, and see if our app is working like Rick wants it to:

Here's the test program, updated to use the new version of Rick's search tool.

```
public class FindGuitarTester {

    public static void main(String[] args) {
        // Set up Rick's guitar inventory
        Inventory inventory = new Inventory();
        initializeInventory(inventory);

        Guitar whatErinLikes = new Guitar("", 0, Builder.FENDER,
            "Stratocaster", Type.ELECTRIC,
            Wood.ALDER, Wood.ALDER);

        List matchingGuitars = inventory.search(whatErinLikes);
        if (!matchingGuitars.isEmpty()) {
            System.out.println("Erin, you might like these guitars:");
            for (Iterator i = matchingGuitars.iterator(); i.hasNext(); ) {
                Guitar guitar = (Guitar)i.next();
                System.out.println("  We have a " +
                    guitar.getBuilder() + " " + guitar.getModel() + " " +
                    guitar.getType() + " guitar:\n    " +
                    guitar.getBackWood() + " back and sides,\n    " +
                    guitar.getTopWood() + " top.\n  You can have it for only $" +
                    guitar.getPrice() + "!\n  ----");
            }
        } else {
            System.out.println("Sorry, Erin, we have nothing for you.");
        }
    }
}
```

We're using enumerated types in this test drive. No typing mistakes this time!

In this new version, we need to iterate over all the choices returned from the search tool.

This time we get a whole list of guitars that match the client's specs.

```
class
FindGui-
tar {
    main()
}
```

FindGuitarTester.java

```
File Edit Window Help SweetSmell
%java FindGuitarTester
Erin, you might like these guitars:
  We have a Fender Stratocaster electric guitar:
    Alder back and sides,
    Alder top.
  You can have it for only $1499.95!
  ----
  We have a Fender Stratocaster electric guitar:
    Alder back and sides,
    Alder top.
  You can have it for only $1549.95!
  ----
```

Everything worked! Erin gets several guitar recommendations, and Rick's customers are going to start buying guitars again.



Yes! That's **exactly** what I want it to do.

Back to our steps

Now that Rick's all set with our software, we can begin to use some OO principles and make sure the app is flexible and well-designed.

✓ **1. Make sure your software does what the customer wants it to do.**

Now that the app does what Rick wants, we're finished up with this step.

2. Apply basic OO principles to add flexibility.

So this is where we can make sure there's no duplicate code, and all our objects are well designed, right?

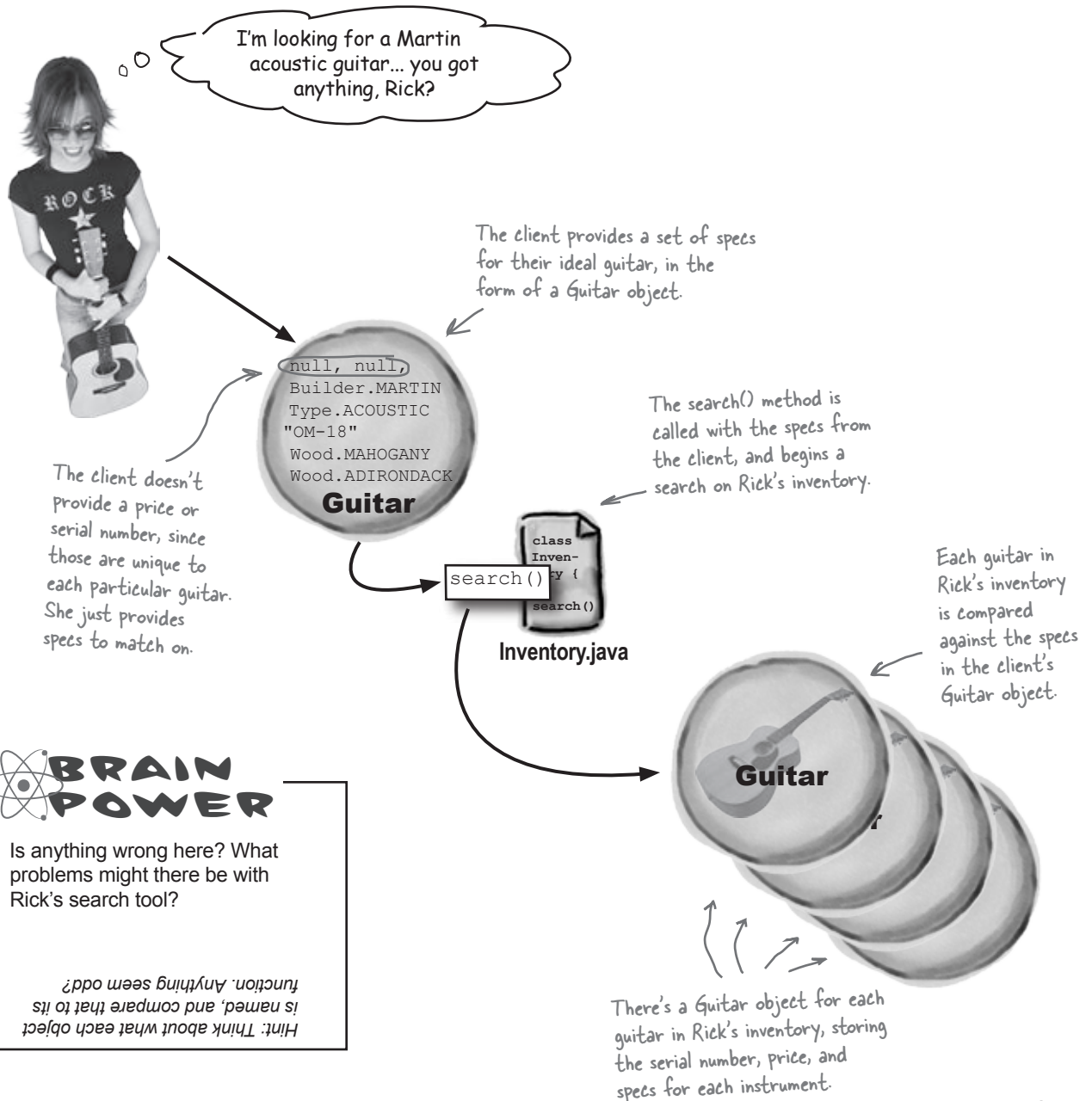
3. Strive for a maintainable, reusable design.

Here's where you take software that works, and make sure the way it's put together actually makes sense.



Looking for problems

Let's dig a little deeper into our search tool, and see if we can find any problems that some simple OO principles might help improve. Let's start by taking a closer look at how the **search()** method in **Inventory** works:



Analyze the `search()` method

Let's spend a little time analyzing exactly what goes on in the `search()` method of `Inventory.java`. Before we look at the code, though, let's think about what this method *should* do.

❶ The client provides their guitar preferences.

Each of Rick's clients has some properties that they're interested in finding in their ideal guitar: the woods used, or the type of guitar, or a particular builder or model. They provide these preferences to Rick, who feeds them into his inventory search tool.

The client can specify only general properties of an instrument. So they never supply a serial number or a price.

❷ The search tool looks through Rick's inventory.

Once the search tool knows what Rick's client wants, it starts to loop through each guitar in Rick's inventory.

❸ Each guitar is compared to the client's preferences.

For each guitar in Rick's inventory, the search tool sees if that guitar matches the client's preferences. If there's a match, the matching guitar is added to the list of choices for the client.

All the general properties, like the top wood and guitar builder, are compared to the client's preferences.

❹ Rick's client is given a list of matching guitars.

Finally, the list of matching guitars is returned to Rick and his client. The client can make a choice, and Rick can make a sale.

Use a textual description of the problem you're trying to solve to make sure that your design lines up with the intended functionality of your application.



The Mystery of the Mismatched Object Type

In the better-designed areas of Objectville, objects are very particular about their jobs. Each object is interested in doing its job, and only its job, to the best of its ability. There's nothing a well-designed object hates more than being used to do something that really isn't its true purpose.

Unfortunately, it's come to our attention that this is exactly what is happening in Rick's inventory search tool: somewhere, an object is being used to do something that it really shouldn't be doing. It's your job to solve this mystery and figure out how we can get Rick's application back in line.

To help you figure out what's gone amiss, here are some helpful tips to start you on your search for the mismatched object type:

1. Objects should do what their names indicate.

If an object is named Jet, it should probably takeOff() and land(), but it shouldn't takeTicket()—that's the job of another object, and doesn't belong in Jet.

2. Each object should represent a single concept.

You don't want objects serving double or triple duty. Avoid a Duck object that represents a real quacking duck, a yellow plastic duck, and someone dropping their head down to avoid getting hit by a baseball.

3. Unused properties are a dead giveaway.

If you've got an object that is being used with no-value or null properties often, you've probably got an object doing more than one job. If you rarely have values for a certain property, why is that property part of the object? Would there be a better object to use with just a subset of those properties?



STOP! Try and solve this mystery before turning the page.

What do you think the mismatched object type is? Write your answer in the blank below:

What do you think you should do to fix the problem? What changes would you make?



You know, Rick's clients really aren't providing a Guitar object... I mean, they don't actually give him a guitar to compare against his inventory.

Frank: Hey, that's right, Joe. I hadn't thought about that before.

Jill: So what? Using a Guitar object makes it really easy to do comparisons in the `search()` method.

Joe: Not any more than some other object would. Look:

```
if (searchGuitar.getBuilder() !=  
    guitar.getBuilder()) {  
    continue;  
}
```

← A small
fragment
from the
`search()`
method in
`Inventory`.

Joe: It really doesn't matter what type of object we're using there, as long as we can figure out what specific things Rick's clients are looking for.

Frank: Yeah, I think we should have a new object that stores just the specs that clients want to send to the `search()` method. Then they're not sending an entire Guitar object, which never seemed to make much sense to me.

Jill: But isn't that going to create some duplicate code? If there's an object for all the client's specs, and then the Guitar has all its properties, we've got two `getBuilder()` methods, two `getBackWood()` methods... that's not good.

Frank: So why don't we just encapsulate those properties away from Guitar into a new object?

Joe: Whoa... I was with you until you said "encapsulate." I thought that was when you made all your variables private, so nobody could use them incorrectly. What's that got to do with a guitar's properties?

Frank: Encapsulation is also about breaking your app into logical parts, and then keeping those parts separate. So just like you keep the data in your classes separate from the rest of your app's behavior, we can keep the generic properties of a guitar separate from the actual Guitar object itself.

Jill: And then Guitar just has a variable pointing to a new object type that stores all its properties?

Frank: Exactly! So we've really encapsulated the guitar properties out of Guitar, and put them in their own separate object. Look, we could do something like this...

Encapsulation
allows you to
hide the inner
workings of your
application's
parts, but yet
make it clear
what each part
does.

↑
New to encapsulation? Flip
ahead to Appendix II, read
that short introduction to
Objectville, and then come
back here and keep reading.